

Fuchsia OS

Руководство разработчика

Перевод официального обучающего гайда [fuchsia.dev](https://fuchsia.dev/fuchsia-src/get-started/learn) (раздел «Get started → Learn the basics») на русский язык: архитектура, ядро Zircon, модель безопасности, компоненты, пакеты, система сборки и интерфейсы FIDL.

Источник: fuchsia.dev/fuchsia-src/get-started/learn — © Google / The Fuchsia Authors.

Текст оригинала распространяется по лицензии Creative Commons Attribution 4.0 (CC BY 4.0); примеры кода — по лицензии Apache 2.0. Это неофициальный перевод, выполненный с помощью Claude (Anthropic); код приведён без изменений из оригинала. Для актуальных данных сверяйтесь с первоисточником.

Собрано: 28.06.2026.

Содержание

Часть I. Концепции и архитектура	3
О Fuchsia: основные принципы	4
Введение в Fuchsia	5
Архитектура	5
Ядро Zircon	6
Песочница и пространства имён	9
Компоненты	13
Пакеты	18
 Часть II. Сборка Fuchsia	 23
Сборка: обзор и лицензии	24
Дерево исходного кода	24
Система сборки (GN/Ninja)	26
Продукты и пакеты сборки	31
Работа с устройствами	35
 Часть III. Компоненты	 38
Компоненты: обзор	39
Объявление компонентов	39
Организация компонентов	43
Тестирование компонентов	48
Отладка компонентов	50
Компоненты на уровне продукта	55
 Часть IV. Интерфейсы FIDL	 56
FIDL: обзор	57
Язык FIDL	57
Соединение компонентов	61
Интеграционное тестирование	66
Диагностика и мониторинг	68

Часть I. Концепции и архитектура

О Fuchsia: основные принципы

Источник: <https://fuchsia.dev/fuchsia-src/concepts>

Узнать о Fuchsia больше можно, погрузившись в концепции, определяющие её устройство. Fuchsia всё ещё быстро развивается, но базовые принципы и ценности системы оставались относительно неизменными на протяжении всего проекта.

Принципы

Ключевые принципы, определяющие проектирование и развитие Fuchsia, — простота, безопасность, обновляемость и производительность.

Простота

Fuchsia упрощает создание, поддержку и интеграцию программного и аппаратного обеспечения для широкого спектра устройств.

- Fuchsia ставит во главу угла ясность и минимализм в архитектуре и дизайне.
- Fuchsia стремится быть простой для понимания и сопровождения.
- Fuchsia предоставляет базовые основы операционной системы — доступ к оборудованию, управление ресурсами и программные абстракции — согласованным образом, создавая надёжную и стабильную платформу для продуктов.

Безопасность

В основе Fuchsia — ядро и программная модель, спроектированные под современные вычисления.

- Система на основе возможностей (capabilities) по умолчанию полностью изолирует процессы и ограничивает доступ программы только теми возможностями и ресурсами, которые были явно предоставлены.
- Программные компоненты распространяются как герметичные пакеты, обеспечивающие границы и гарантии безопасности.
- Изоляцию ПО («песочницу») обеспечивает операционная система, что снижает ресурсные затраты на безопасность.

Обновляемость

Будучи модульной операционной системой, Fuchsia позволяет независимо обновлять ядро, драйверы и программные компоненты.

- Стабильные ABI позволяют операционной системе, драйверам устройств и продуктовому ПО сохранять совместимость на длинных временных горизонтах. Это снижает нагрузку по сопровождению на разработчиков продуктов и продлевает срок службы устройств для потребителей.
- ПО поставляется в пакетах, которые можно обновлять независимо и даже доставлять по требованию, как в вебе.
- Google намерен обновлять и поддерживать Fuchsia со временем.

Производительность

Fuchsia спроектирована под требования реальных продуктов и оптимизирована по производительности.

- Fuchsia — операционная система общего назначения, обеспечивающая высокую производительность на различных платформах, архитектурах и устройствах.
- Fuchsia эффективно управляет системными ресурсами — процессорами, памятью, хранилищем, сетью и питанием — для оптимизации производительности.
- Fuchsia достигает целевых показателей производительности в коммерческих устройствах, уже находящихся в производстве.

Введение в Fuchsia

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/intro>

Fuchsia — новая операционная система с открытым исходным кодом, созданная в Google «с нуля», начиная с ядра, чтобы отвечать потребностям растущей экосистемы подключённых устройств.

В этом учебном модуле вы изучите ключевые принципы, лежащие в основе новой ОС, и узнаете, как Fuchsia создаёт фундамент для разработчиков, позволяющий создавать долгоживущие продукты и сценарии на широком спектре устройств.

Что вы узнаете

- Архитектуру Fuchsia и ключевые принципы проектирования
- Модель безопасности и программной изоляции Fuchsia
- Как ПО упаковывается и доставляется в Fuchsia

Что вам понадобится

- Рабочая машина под управлением Linux
- Эмулятор Fuchsia (FEMU)
- Успешно собранная Fuchsia с конфигурацией `fx set workbench_eng.x64`

Архитектура

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/intro/architecture>

Проектирование и развитие Fuchsia направляют следующие архитектурные принципы:

- Простота: Fuchsia упрощает создание, поддержку и интеграцию ПО и оборудования для широкого спектра устройств.
- Безопасность: в основе Fuchsia — ядро и программная модель, спроектированные под современные вычисления.
- Обновляемость: будучи модульной ОС, Fuchsia позволяет независимо обновлять ядро, драйверы и программные компоненты.

- Производительность: Fuchsia спроектирована под требования реальных продуктов и оптимизирована по производительности.

Ядро системы — Zircon, ядро ОС и набор библиотек для запуска и начальной загрузки системы. Все остальные системные компоненты реализованы в пользовательском пространстве и изолированы, что усиливает принцип наименьших привилегий. Сюда входят:

- Драйверы устройств
- Файловые системы
- Сетевые стеки

Ядро Zircon

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/intro/zircon>

Zircon — это ядро, на котором работает Fuchsia. Он состоит из ядра ОС и небольшого набора пользовательских служб, драйверов и библиотек, необходимых для базовых системных функций, таких как загрузка.

Хотя Zircon применяет многие идеи, популяризированные микроядрами, он не стремится быть минимальным. Вместо этого микроядерная по духу архитектура Zircon позволяет Fuchsia сократить объём доверенного кода в системе до нескольких базовых функций:

- Управление памятью
- Планирование
- Межпроцессное взаимодействие

Системные вызовы

Код пользовательского пространства взаимодействует с объектами в пространстве ядра через системные вызовы. У Zircon есть системные вызовы для низкоуровневых операций, таких как:

- Управление памятью
- Управление задачами и процессами
- Межпроцессное взаимодействие (IPC) и синхронизация
- Обработка исключений
- Службы аппаратной поддержки (часы, энтропия, ввод-вывод устройств)

У Zircon меньше системных вызовов, чем у POSIX-ориентированных ОС, поскольку службы вроде файловых систем и драйверов размещаются вне ядра. Полный список доступных системных вызовов Zircon см. в справочной документации.

Пользовательские процессы обращаются к системным вызовам через `libzircon.so` — виртуальный динамический разделяемый объект (vDSO). Zircon vDSO — это разделяемая библиотека в формате ELF, которую ядро отображает в адресное пространство каждого нового процесса. Библиотека считается «виртуальной», потому что предоставляется непосредственно образом ядра, а не загружается из файла.

Большинство системных вызовов работают с одним или несколькими дескрипторами (handle) — локальными для процесса ссылками на объекты в пространстве ядра, представленными 32-битным целым (zx_handle_t). Каждый дескриптор объявляет привилегии, или права (rights), которыми обладает владелец для выполнения действий над самим дескриптором или объектом, на который он ссылается.

Подобно файловым дескрипторам POSIX, дескрипторы Zircon — это ссылки на конкретный объект ядра, и они участвуют в предоставлении возможностей. Однако дескрипторы Zircon чуть гибче: права применяются к дескриптору, а не к вызывающему процессу. Один процесс может иметь два разных дескриптора на один и тот же объект ядра с разными правами.

Кроме того, на дескрипторы нельзя ссылаться по имени, и Zircon не резервирует идентификаторы для распространённых потоков вроде stdin и stdout.

Подробнее см. «Дескрипторы Zircon».

Задания (jobs), процессы и потоки

Zircon предоставляет три основных объекта ядра для выполнения кода:

- Поток (thread): поток выполнения в рамках заданного адресного пространства.
- Процесс (process): набор исполняемых инструкций, выполняемых в приватном изолированном адресном пространстве.
- Задание (job): группа связанных процессов и заданий. Все задания образуют единое дерево с общим корнем.

Процессы лежат в основе системных возможностей. Каждому процессу предоставляется набор возможностей через дескрипторы, которыми он владеет.

ПО Fuchsia может выполняться как в рамках одного процесса, так и нет. Задания позволяют управлять «приложениями», состоящими более чем из одного процесса, как единым целым.

Межпроцессное взаимодействие

Поскольку процессы по умолчанию изолированы, ядро должно обеспечить им способ безопасно взаимодействовать друг с другом. Zircon включает следующие типы объектов ядра для межпроцессного взаимодействия (IPC):

- Event (событие): интерфейс сигнализации между двумя процессами.
- Socket (сокет): потоковый транспорт данных, похожий на канал (pipe).
- Stream (поток): потоковый транспорт данных с возможностью перемещения позиции, как в файле.
- Channel (канал): транспорт на основе сообщений, способный передавать как данные, так и набор дескрипторов.
- FIFO: управляющий канал для доступа к разделяемой памяти, оптимизированный под небольшие полезные нагрузки.

Среди этих объектов каналы особенно хорошо подходят для запуска новых процессов, поскольку способны передавать дескрипторы (а значит, и возможности) другому процессу.

У канала ровно два конечных дескриптора, каждым из которых владеет отдельный процесс. Читать и писать сообщения могут только владельцы, но владение концом канала можно передать от одного процесса другому. Когда дескрипторы записываются в канал, они удаляются из отправляющего процесса. Когда сообщение с дескрипторами читается из канала, дескрипторы добавляются в принимающий процесс.

Каналы Zircon — основа сервисных IPC-протоколов, описываемых языком определения интерфейсов Fuchsia (FIDL). FIDL-протоколы — основной метод IPC, используемый программами Fuchsia. Создание и потребление FIDL-протоколов вы рассмотрите подробнее позже.

Упражнение: задания и процессы

Давайте исследуем некоторые из этих фундаментальных концепций на работающей системе. В этом упражнении вы увидите, как задания и процессы образуют дерево.

Запуск эмулятора

Если у вас ещё нет запущенного экземпляра, запустите эмулятор:

```
ffx emu start --headless
```

По завершении запуска эмулятор печатает следующее сообщение и возвращает управление:

```
Logging to "$HOME/.local/share/Fuchsia/ffx/emu/instances/fuchsia-emulator/emulator.log"
Waiting for Fuchsia to start (up to 60 seconds).....
Emulator is ready.
```

Вывод списка процессов

Подключитесь к оболочке устройства и используйте команду `ps`, чтобы вывести список выполняемых заданий и процессов.

```
fx shell ps
```

Ниже — сокращённый пример того, как выглядит вывод:

TASK	PSS	PRIVATE	SHARED	STATE	NAME
j: 1027	507.8M	507.4M			root
p: 1061	564.4k	564k	36k		bin/bootsvc
p: 1150	4264.4k	4264k	36k		bin/component_manager
j: 1479	228.4k	228k			
p: 1583	228.4k	228k	36k		pwrbtn-monitor.cm
j: 1484	532.4k	532k			
p: 1599	532.4k	532k	36k		svchost.cm
j: 1544	402.4k	304k			
p: 1633	402.4k	304k	232k		netsvc.cm
j: 1681	296.4k	296k			
p: 1733	296.4k	296k	36k		console-launcher.cm
j: 1799	7232.4k	7232k			
p: 1825	7232.4k	7232k	36k		archivist.cm
j: 1927	660.4k	660k			
p: 1955	660.4k	660k	36k		base-resolver.cm
j: 2072	1016.4k	1016k			
p: 2088	1016.4k	1016k	36k		driver_manager.cm
j: 2239	348.4k	348k			
p: 2252	348.4k	348k	36k		device-name-provider.cm
j: 2364	275.3M	275.3M			
p: 2380	1012.4k	1012k	36k		fshost.cm
p: 6544	252.1M	252.1M	36k		/pkg/bin/blobfs
p: 10205	9744.4k	9744k	36k		/pkg/bin/minfs
p: 10475	12.8M	12.8M	36k		pkgfs

Сосредоточимся пока на двух столбцах вывода:

- TASK: показывает, является ли запись заданием (j) или процессом (p), за чем следует их уникальный идентификатор.
- NAME: даёт чуть больше деталей о том, какая часть системы там выполняется.

Разберём несколько интересных моментов на основе уже обсуждённого:

- Каждый процесс связан с родительским заданием. У некоторых заданий несколько процессов.
- Все задания восходят к корневому заданию как к предельному родителю, образуя дерево.
- При запуске система помещает несколько процессов напрямую в корневое задание. Большинство остальных запускаются под собственными родительскими заданиями.
- После начальной загрузки многие записи имеют расширение .cm. Они относятся к компонентам, о которых вы узнаете позже.
- Некоторые из этих компонентов — базовые службы, такие как файловые системы (fshost.cm) и драйверы (driver_manager.cm), которые живут в пользовательском пространстве отдельно от ядра.

Далее мы разберём, как Zircon обеспечивает основы модели безопасности Fuchsia.

Изучение дерева выполняемых процессов — отличный способ узнать о процессе запуска устройства Fuchsia. Уделите время обзору запуска устройства и сопоставьте начальные процессы с выводом ps в эмуляторе.

Песочница и пространства имён

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/intro/sandboxing>

В этом разделе вы узнаете, как объекты ядра Zircon позволяют Fuchsia следовать принципу наименьших привилегий, изолируя процессы и предоставляя им только необходимые возможности.

Песочница (sandboxing)

При создании нового процесса у него нет никаких возможностей. Процесс полностью зависит от своего создателя, который предоставляет возможности через набор переданных дескрипторов. Можно сказать, что пустой процесс не обладает «эмбиентными» полномочиями (ambient authority).

Поэтому процессы обычно создаются с некоторым начальным набором ресурсов и возможностей. Протокол `fuchsia.process.Launcher` предоставляет низкоуровневый интерфейс для создания новых процессов из исполняемого файла и набора дескрипторов объектов ядра. Большинство ПО использует фреймворк компонентов, упрощающий настройку нового процесса со стандартным набором начальных возможностей. Компоненты вы рассмотрите подробнее позже.

Ранее вы видели, что дескрипторы — это уникальные ссылки на объекты ядра. Каждый дескриптор также содержит права на выполнение определённых действий, такие как `ZX_RIGHT_READ`, `ZX_RIGHT_WRITE` или `ZX_RIGHT_EXECUTE`.

При создании процесса права каждого дескриптора можно сократить под требования (и ограничения) нового процесса с помощью операций `zx_handle_replace()` или `zx_handle_duplicate()`. Затем создающий процесс может записать новые дескрипторы в IPC-канал, задав начальные возможности нового процесса.

Некоторые начальные дескрипторы, передаваемые процессу, — это каталоги, которые процесс монтирует в своё пространство имён.

Пространства имён (namespaces)

Пространство имён процесса содержит его приватное представление о мире и определяет, на какую часть системы Fuchsia процесс может влиять. По сути, это и есть правила песочницы, в которой работает процесс.

Пространства имён наполняются различными ресурсными объектами, включая:

- **Files (файлы):** объекты, содержащие двоичные данные.
- **Directories (каталоги):** объекты, содержащие другие объекты.
- **Sockets (сокет):** объекты, устанавливающие соединение при открытии, как именованные каналы.
- **Protocols и services (протоколы и службы):** объекты, предоставляющие структурированные службы при открытии.
- **Devices (устройства):** объекты, дающие доступ к аппаратным ресурсам.

Создатель процесса наполняет пространство имён исходя из набора требуемых возможностей. Процесс не может добавлять объекты в собственное пространство имён, поскольку это означало бы, что он сам себе предоставляет возможности доступа к этим объектам.

Во многом содержимое пространства имён напоминает файловые ресурсы POSIX-ориентированных ОС, где «всё есть файл». Однако есть важные отличия. Пространства имён определяются для каждого процесса, и, в отличие от других ОС, у Fuchsia нет «корневой файловой системы». Вместо этого путь / указывает на корень приватного пространства имён процесса. Это также означает, что в Fuchsia нет понятия chroot-окружений, так как у каждого процесса фактически свой приватный «корень». Это влияет и на обход каталогов, и на то, как файловые серверы разрешают пути с ../. Подробнее см. «dot-dot considered harmful».

Упражнение: пространства имён

Большинство процессов в Fuchsia представляют исполняемые программы, связанные с компонентом, причём именно объявление компонента отвечает за построение пространства имён, доступного процессу.

В этом упражнении вы исследуете содержимое пространства имён компонента.

Запуск эмулятора

Если у вас ещё нет запущенного экземпляра, запустите эмулятор:

```
ffx emu start --headless
```

По завершении запуска эмулятор печатает следующее сообщение и возвращает управление:

```
Logging to "$HOME/.local/share/Fuchsia/ffx/emu/instances/fuchsia-emulator/emulator.log"
Waiting for Fuchsia to start (up to 60 seconds).....
Emulator is ready.
```

Поиск целевого компонента

В предыдущем разделе вы узнали, что процессы, связанные с компонентом, именуются с расширением .cm. Вспомните следующий пример списка процессов:

TASK	PSS	PRIVATE	SHARED	STATE	NAME
j: 1027	507.8M	507.4M			root
p: 1061	564.4k	564k	36k		bin/bootsvc
p: 1150	4264.4k	4264k	36k		bin/component_manager
j: 1479	228.4k	228k			
p: 1583	228.4k	228k	36k		pwrbtn-monitor.cm
j: 1484	532.4k	532k			
p: 1599	532.4k	532k	36k		svchost.cm
j: 1544	402.4k	304k			
p: 1633	402.4k	304k	232k		netsvc.cm
j: 1681	296.4k	296k			
p: 1733	296.4k	296k	36k		console-launcher.cm
j: 1799	7232.4k	7232k			
p: 1825	7232.4k	7232k	36k		archivist.cm
...					
j: 31294	1872.2K	1872K			
p: 31331	1872.2K	1872K	20K		http-client.cm

В этом упражнении в качестве цели для исследования вы будете использовать http-client.cm.

Подключение к целевому компоненту

Чтобы исследовать пространство имён компонента, нужно определить его уникальный идентификатор в системе. Он называется `moniker` (моникер) компонента.

Используйте команду `ffx component show`, чтобы вывести дополнительные сведения о компоненте, включая его `moniker`:

```
ffx component show http-client.cm
```

Команда печатает вывод, похожий на следующий:

```
$ ffx component show http-client.cm
    Moniker: /core/network/http-client
      URL: #meta/http-client.cm
    Type: CML static component
    ...
```

Командой `ffx component explore` можно открыть интерактивную оболочку внутри окружения целевого компонента. Попробуйте это для компонента `http-client`:

```
ffx component explore /core/network/http-client
```

Внутри оболочки `explore` выведите содержимое корневого каталога командой `ls`:

```
[explore shell] $ ls
```

```
bin
exposed
ns
out
runtime
svc
```

Исследование пространства имён

Пространство имён компонента находится по пути `/ns` внутри окружения.

Внутри оболочки `explore` выведите содержимое пространства имён:

```
[explore shell] $ ls /ns
```

```
config
pkg
svc
```

Краткое описание каждого элемента:

- `config/`: данные конфигурации компонента
- `pkg/`: содержимое пакета компонента
- `svc/`: системные службы, доступные компоненту

Внутри оболочки `explore` выведите содержимое входящего каталога `/ns/svc`. Этот каталог содержит узлы служб, представляющие системные службы, предоставленные данному компоненту.

```
[explore shell] $ ls /ns/svc
```

```
fuchsia.logger.LogSink  
fuchsia.net.name.Lookup  
fuchsia.posix.socket.Provider
```

Каждая из этих служб доступна по общеизвестному протоколу, определённому интерфейсом на языке FIDL. Протоколы FIDL и доступ к различным службам мы рассмотрим подробнее позже.

Внутри оболочки `explore` введите `exit`, чтобы вернуться в оболочку на вашей рабочей машине:

```
[explore shell] $ exit
```

Прочитайте руководство «Explore components», чтобы узнать о других элементах каталога в окружении этого компонента. Затем подключитесь к другому целевому компоненту. Чем отличается содержимое у двух разных компонентов?

Компоненты

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/intro/components>

Компоненты — это фундаментальные строительные блоки ПО, работающего в Fuchsia. Каждый компонент — это компонуемый модуль в песочнице, который взаимодействует с другими компонентами через возможности (capabilities). Это повышает безопасность системы и создаёт чёткие интерфейсы между компонентами, упрощая их обновление и замену.

В Fuchsia (почти) всё является компонентом. Вспомните из обсуждения Zircon, что поверхность ядра намеренно мала, а большинство базовых служб реализованы в пользовательском пространстве. Это значит, что большая часть ПО, работающего в Fuchsia, реализована с помощью фреймворка компонентов, включая:

- Пользовательские приложения
- Драйверы устройств
- Файловые системы
- Медиакодеки
- Сетевые стеки

За пределами ядра есть лишь несколько низкоуровневых исключений, не использующих фреймворк компонентов, например загрузчики (bootloaders) и процесс `userboot`.

Менеджер компонентов (component manager)

Сердце фреймворка компонентов — менеджер компонентов. Он отвечает за координацию выполнения всех экземпляров компонентов, предоставление им возможностей и посредничество в соединениях между компонентами.

Компоненты могут запускаться явно (например, по URL) или неявно — по запросу определённой возможности. Менеджер компонентов выполняет необходимое разрешение, чтобы решить, запускать ли новый компонент или направить запрос

существующему экземпляру. Чтобы такая маршрутизация была возможна, каждый компонент должен объявлять как возможности, которые он предоставляет системе, так и те, которые он потребляет.

Zircon определяет общие объекты ядра для исполняемого кода, например процессы. Однако экземпляры компонентов не всегда соответствуют одному процессу. Часто политику использования этих процессов задаёт runner. Например, ELF-runner запускает каждый компонент в новом задании с процессом, выполняющим исполняемый код.

Больше примеров см. в «компоненты против процессов».

Менеджер компонентов разбирает объявление каждого компонента, чтобы понять, как его запускать и какие возможности предоставить. Обычно компоненты объявляются системе через файл манифеста компонента внутри пакета компонента.

Ниже — простой пример манифеста компонента, описывающего ELF-исполняемый файл с дополнительными аргументами командной строки:

```
program: {
  runner: "elf",
  binary: "bin/hello",
  args: [ "Hello", "World!" ],
},
```

Обратите внимание на объявление среды выполнения (runtime), сообщающее менеджеру компонентов, что этому компоненту требуется ELF-runner. Это пример возможности!

Возможности компонентов

Компоненты получают привилегии на доступ к различным частям системы через возможности (capabilities). Каждый компонент может объявлять новые возможности, которые он предлагает системе, и возможности, предоставляемые другими компонентами (или фреймворком), которые нужны ему для работы.

Как вы только что видели, runner — пример возможности, объявляющей среду выполнения компонента. Другие распространённые типы возможностей — directory для доступа к файловым ресурсам и protocol для связи с другими компонентами.

Разработчики объявляют типы возможностей, необходимых компоненту, в манифесте компонента. Ниже — пример манифеста, запрашивающего две возможности: доступ на чтение к каталогу example-data и службу, описанную FIDL-протоколом fuchsia.example.Foo.

```
use: [
  {
    directory: "example-data",
    rights: [ "r*" ],
    path: "/example/data",
  },
  {
    protocol: "fuchsia.example.Foo",
  },
]
```

Менеджер компонентов использует объявления возможностей, чтобы заполнить пространство имён каждого компонента нужными дескрипторами каталогов. В этом примере компонент получил бы в своём пространстве имён `/example/data` и `/svc/fuchsia.example.Foo`.

Организация компонентов

Все компоненты системы образуют единое дерево экземпляров компонентов с общим корнем. Эта древовидная структура определяет несколько важных аспектов поведения компонентов.

Родительские компоненты в дереве отвечают за создание экземпляров других компонентов как своих детей и предоставление им необходимых возможностей. В то же время дочерние компоненты могут экспонировать (`expose`) возможности обратно родителю. Дочерние компоненты можно создавать двумя способами:

- Статически: родитель объявляет существование ребёнка в собственном объявлении компонента.
- Динамически: родитель добавляет ребёнка в коллекцию компонентов во время выполнения, используя протокол `fuchsia.component.Realm`.

Любой родительский компонент со всеми его детьми образует группу в дереве, называемую `realm` (область). `Realm` позволяет родителю контролировать, какие возможности входят в его поддерево компонентов и выходят из него, создавая границу возможностей. Компоненты решают, экспонировать ли возможности за пределы своего `realm`, с помощью ключевого слова `expose`:

```
expose: [  
  {  
    protocol: "fuchsia.example.Foo",  
    from: "self",  
  },  
],
```

После того как возможность экспонирована в `realm`, родитель может поделиться ею с другими компонентами того же `realm`. Это делается с помощью ключевого слова `offer`:

```
offer: [  
  {  
    protocol: "fuchsia.example.Foo",  
    from: "self",  
  },  
],
```

Менеджер компонентов отвечает за сопоставление запросов на доступ к возможности (например, каталогу или протоколу) с компонентом, предоставляющим эту возможность. Это называется маршрутизацией возможностей (`capability routing`). Менеджер может разрешать только те возможности, которые экспонированы и предложены в пределах одного `realm`.

Упражнение: компоненты

В этом упражнении вы исследуете дерево экземпляров компонентов и подробно рассмотрите маршрутизацию возможностей в действии на примере нескольких базовых системных компонентов.

Запуск эмулятора

Если у вас ещё нет запущенного экземпляра, запустите эмулятор:

```
ffx emu start --headless
```

По завершении запуска эмулятор печатает следующее сообщение и возвращает управление:

```
Logging to "$HOME/.local/share/Fuchsia/ffx/emu/instances/fuchsia-emulator/emulator.log"
Waiting for Fuchsia to start (up to 60 seconds).....
Emulator is ready.
```

Исследование системных компонентов

Откройте другое окно терминала и командой `component list` выведите дерево компонентов системы:

```
ffx component list
```

Вы должны увидеть вывод, похожий на (сокращённый) список ниже:

```
/
/bootstrap
/bootstrap/archivist
/bootstrap/base_resolver
/bootstrap/console
/bootstrap/console-launcher
/bootstrap/decompressor
/bootstrap/device_name_provider
/bootstrap/driver_manager
/bootstrap/fshost
/bootstrap/kernel_debug_broker
/bootstrap/miscsvc
/bootstrap/netsvc
/bootstrap/power_manager
/bootstrap/ptysvc
/bootstrap/pwrbtn-monitor
/bootstrap/shutdown_shim
/bootstrap/sysinfo
/bootstrap/virtual_console
/core
/core/activity
...
/core/detect
/core/font_provider
/core/log-stats
/core/remote-control
/core/sampler
/core/system-update-commmitter
/core/temperature-logger
/core/test_manager
/startup
```

Этот список представляет дерево экземпляров компонентов, где организационные компоненты — `bootstrap`, `core`, `startup` — образуют поддеревья под корнем.

Команда `component show` даёт больше деталей о каждом компоненте.

Используйте её, чтобы посмотреть детали `http-client` — компонента, предоставляющего службу HTTP-запросов:

```
ffx component show http-client.cm
```

Команда выводит следующий отчёт:

```
Moniker: /core/network/http-client
URL: #meta/http-client.cm
Type: CML static component
Component State: Resolved
Incoming Capabilities: config
                      fuchsia.logger.LogSink
                      fuchsia.net.name.Lookup
                      fuchsia.posix.socket.Provider
                      pkg
Exposed Capabilities: fuchsia.net.http.Loader
Merkle root: d9e73f5b061f2f227e596e2e0079ff3a095fc69e192cf85e0d7621826c76356c
Execution State: Running
Start reason: '/core/feedback' requested capability 'fuchsia.net.http.Loader'
Running since: ...
Job ID: 41268
Process ID: 41311
Outgoing Capabilities: fuchsia.net.http.Loader
```

Обратите внимание на несколько деталей в отчёте:

- Уникальный идентификатор экземпляра компонента (называемый `moniker`).
- URL пакета, откуда был загружен компонент.
- Состояние выполнения компонента.
- Текущий идентификатор задания/процесса, где выполняется экземпляр.
- Набор запрашиваемых и экспонируемых возможностей компонента.

Трассировка маршрута возможности

В предыдущем выводе перечислены три группы возможностей:

- **Incoming Capabilities** (входящие): возможности, которые компонент объявляет через `use`. Они предоставляются компоненту через его пространство имён.
- **Outgoing Capabilities** (исходящие): возможности, которые компонент опубликовал в свой исходящий каталог (`outgoing directory`).
- **Exposed Capabilities** (экспонированные): возможности, которые компонент объявляет через `expose`. Это экспонированные службы компонента.

Одна из возможностей, экспонируемых `http-client` его родительскому `realm`, — `fuchsia.net.http.Loader`. Она позволяет другим компонентам выполнять HTTP-запросы.

Командой `component capability` определите, сколько компонентов взаимодействуют с этой возможностью:

```
ffx component capability fuchsia.net.http.Loader
```

Команда выводит все подходящие компоненты:

```
Exposed:
  /core/network/http-client
  /core/network
Used:
  /core/cobalt
  /core/feedback
  /core
```

Это означает, что компоненты `cobalt` и `feedback` используют эту возможность (т. е. она указана у них в `Incoming Capabilities`). Общий предок этих компонентов — `core`, который и выполняет маршрутизацию этой возможности к нужным детям.

Многие компоненты используют возможность `fuchsia.logger.LogSink`, необходимую для чтения системных логов. Их можно перечислить тем же поиском `component capability` по входящим возможностям.

Сможете ли вы найти, какой компонент экспонирует эту возможность?

Пакеты

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/intro/packages>

ПО Fuchsia доставляется в систему по требованию через пакеты. Это критически важный элемент для ключевых принципов Fuchsia — безопасности и обновляемости. Пакеты можно обновлять независимо и доставлять по требованию, как веб-страницу. Это позволяет выпустить патч уязвимости сразу для всех продуктов Fuchsia без индивидуальной координации по каждому продукту.

Пакет — это не один архив или файл образа, а дерево больших двоичных объектов (BLOB). Корень дерева — BLOB под названием `meta.far`, который содержит метаданные пакета, включая файл `meta/contents` со ссылками на остальные BLOB-объекты. BLOB-объекты внутри пакетов Fuchsia адресуются по содержимому (`content-addressed`), то есть на них ссылаются по хешу их содержимого. Адрес содержимого самого `meta.far` называется хешем пакета (`package hash`).

Адреса содержимого вычисляются как корневой хеш дерева Меркла. Это распространённый приём для проверки целостности содержимого при передаче и на диске. Он позволяет Fuchsia проверить содержимое всего пакета, используя лишь хеш пакета!

Подробнее о корнях Меркла и их применении в Fuchsia см. «Fuchsia package hash». `meta.far` содержит каталог `meta/` как минимум со следующими двумя элементами:

- `meta/package`: JSON-файл с информацией об идентичности пакета, такой как имя и версия.
- `meta/contents`: отображение человекочитаемых имён файлов пакета на их адреса содержимого.

Если два или более пакетов используют одно и то же содержимое (например, библиотечную зависимость или ресурс шрифта), их метаданные будут указывать на один и тот же адрес содержимого для этого ресурса. Это позволяет Fuchsia оптимизировать распространение и хранение пакетов, не загружая и не сохраняя повторно BLOB, который уже существует.

Размещение и раздача пакетов

Пакеты размещаются в репозиториях на основе The Update Framework (TUF). Это спецификация, разработанная для безопасной доставки обновлений ПО.

Репозитории TUF защищают обновления подписанными метаданными, которые проверяются относительно известных доверенных открытых и закрытых ключей. Это значит, что раздавать репозиторий TUF может любой HTTP-сервер без транспортной защиты, включая рабочую станцию разработчика!

Пакеты в репозитории идентифицируются URL со схемой fuchsia-pkg:

```
fuchsia-pkg://repo-hostname/pkg-name#resource-path
```

- repo-hostname: имя хоста доверенного репозитория пакетов, например fuchsia.com.
- pkg-name: уникальный идентификатор пакета в этом репозитории.
- resource-path: ресурс внутри пакета, например манифест компонента.

Запросы на ПО на устройстве Fuchsia обрабатывает разрешитель пакетов (package resolver). Он определяет, есть ли пакет уже в локальном кеше системы. Если нет, разрешитель загружает meta.far из репозитория и обновляет необходимые BLOB-объекты содержимого.

Хранение пакетов

На устройстве BLOB-объекты пакетов хранятся в файловой системе с адресацией по содержимому, оптимизированной под файлы «записал один раз — читаешь часто», — blobfs. Это позволяет дедуплицировать их по всем пакетам и криптографически проверять по хешу. Поверх blobfs Fuchsia запускает службу pkg-cache для управления пакетами.

Уровень pkg-cache отслеживает, какие пакеты в системе сейчас активны. В Fuchsia пакеты не устанавливаются и не удаляются явно. ПО доставляется по требованию, и так же место может высвободиться от неактивных пакетов через периодическую сборку мусора. Когда pkg-cache запускает сборку мусора, BLOB-объекты содержимого, на которые не ссылается ни один активный пакет, удаляются.

Упражнение: пакеты

До сих пор в этом codelab вы уже пользовались доставкой ПО по требованию и, вероятно, даже не подозревали об этом! В этом упражнении вы заглянете под капот и увидите, как пакеты доставляются и хранятся на устройстве Fuchsia.

Перезапуск эмулятора

- Выполните следующую команду, чтобы закрыть любые открытые экземпляры эмулятора: `ffx emu stop --all`
- Запустите новый экземпляр эмулятора: `ffx emu start --headless`. По завершении запуска эмулятор печатает сообщение о готовности и возвращает управление.

Запуск локального сервера пакетов

Выполните следующую команду, чтобы запустить сервер пакетов и позволить эмулятору загружать программные пакеты:

```
fx serve
```

Команда печатает вывод, похожий на следующий, указывая, что сервер запущен и успешно зарегистрировал эмулятор как целевое устройство:

```
[serve] Discovery...  
[serve] Device up  
[serve] Registering devhost as update source  
[serve] Ready to push packages!  
[serve] Target uptime: 139  
[pm auto] adding client: [fe80::5888:cea3:7557:7384%qemu]:46126  
[pm auto] client count: 1
```

Изучение сервера пакетов

Команда `fx serve` запускает локальный сервер пакетов для доставки пакетов на целевые устройства. По умолчанию этот сервер работает на порту 8083.

Откройте в браузере <http://localhost:8083>. Загрузится HTML-страница со списком всех пакетов, доступных сейчас в репозитории. Каждый из них может быть доставлен на устройство.

Наблюдение за загрузкой пакетов

Пакеты разрешаются и загружаются устройством Fuchsia по требованию. Посмотрим на это в действии на примере пакета `spinning-square`.

В оболочке устройства можно проверить, находится ли известный пакет сейчас на устройстве:

```
fx shell pkgctl pkg-status fuchsia-pkg://fuchsia.com/spinning-square-rs
```

```
Package in registered TUF repo: yes (merkle=ef65e2ed...)  
Package on disk: no
```

Откройте новый терминал и начните потоковый просмотр логов устройства для `pkg-resolver`:

```
ffx log --filter pkg-resolver
```

Это показывает все случаи, когда пакет загружался с сервера пакетов.

В оболочке устройства попробуйте разрешить пакет:

```
ffx target package resolve fuchsia-pkg://fuchsia.com/spinning-square-rs
```

Обратите внимание на новые строки в выводе логов `pkg-resolver`:

```
[pkg-resolver] INFO: attempting to resolve fuchsia-pkg://fuchsia.com/spinning-square-rs as  
  fuchsia-pkg://default/spinning-square-rs with TUF  
[pkg-resolver] INFO: resolved fuchsia-pkg://fuchsia.com/spinning-square-rs as  
  fuchsia-pkg://default/spinning-square-rs to  
  21967ecc643257800b8ca14420c7f023c1ede7a76068da5faedf328f9d9d3649 with TUF
```

В оболочке устройства снова проверьте статус пакета на устройстве:

```
fx shell pkgctl pkg-status fuchsia-pkg://fuchsia.com/spinning-square-rs
```

```
Package in registered TUF repo: yes (merkle=21967ecc...)
Package on disk: yes
```

Fuchsia разрешила пакет и загрузила его из локального репозитория TUF по требованию!

Изучение метаданных пакета

Теперь, когда пакет `spinning-square` успешно разрешён, можно исследовать его содержимое. После разрешения пакет на целевом устройстве адресуется по своему адресу содержимого.

В оболочке устройства командой `pkgctl get-hash` определите хеш пакета `spinning-square`:

```
fx shell pkgctl get-hash fuchsia-pkg://fuchsia.com/spinning-square-rs
```

Команда возвращает уникальный хеш пакета:

```
ef65e2ed...
```

Передайте полный хеш пакета команде `pkgctl open`, чтобы просмотреть содержимое пакета:

```
fx shell pkgctl open ef65e2ed...
```

```
opening ef65e2ed...
package contents:
/bin/spinning_square
/lib/VkLayer_khronos_validation.so
/lib/ld.so.1
/lib/libasync-default.so
/lib/libbackend_fuchsia_globals.so
/lib/libc++.so.2
/lib/libc++abi.so.1
/lib/libfdio.so
/lib/librust-trace-provider.so
/lib/libstd-e3c06c8874beb723.so
/lib/libsyslog.so
/lib/libtrace-engine.so
/lib/libunwind.so.1
/lib/libvulkan.so
/meta/contents
/meta/package
/meta/spinning-square-rs.cm
/data/fonts/RobotoSlab-Regular.ttf
/meta/fuchsia.abi/abi-revision
/data/vulkan/explicit_layer.d/VkLayer_khronos_validation.json
```

Это перечисляет метаданные пакета и каждый BLOB содержимого в пакете. Видны записи `bin/` для исполняемых файлов, `lib/` для зависимостей разделяемых библиотек, а также дополнительные метаданные и ресурсы.

Что дальше?

Поздравляем! Теперь вы лучше понимаете, что делает Fuchsia уникальной и какие цели движут проектированием этой новой платформы.

В следующем модуле вы узнаете больше об открытом проекте Fuchsia и инструментах, используемых для сборки и настройки системы:

Сборка Fuchsia

Часть II. Сборка Fuchsia

Сборка: обзор и лицензии

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/build>

Fuchsia — открытый проект, стремящийся быть инклюзивным. Google задаёт направление Fuchsia и принимает решения о платформе, но проект поощряет вклад со стороны всего сообщества. Баг-трекер и списки рассылки публичны, а открытый процесс RFC обеспечивает последовательный и прозрачный путь принятия общепроектных технических решений.

Код Fuchsia покрыт несколькими открытыми лицензиями:

- Ядро Zircon выпускается под лицензией в стиле MIT.
- Компоненты пользовательского пространства Fuchsia выпускаются под лицензией в стиле BSD.

Любой код в составе проекта с другим правообладателем или под другой лицензией считается внешним (external) кодом.

Вклад в открытый проект начинается с понимания того, как ориентироваться в коде и собирать его. В этом учебном модуле вы подробнее узнаете об открытом дереве исходников Fuchsia и успешно выполните свою первую сборку продукта Fuchsia.

Предварительные требования

- Введение в Fuchsia

Что вы узнаете

- Навигацию по дереву исходников Fuchsia и его основным компонентам
- Конфигурации сборки Fuchsia и их настройку
- Работу с Fuchsia на устройствах

Что вам понадобится

- Рабочая машина под управлением Linux
- Настроенное окружение сборки с исходным кодом Fuchsia

Дерево исходного кода

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/build/source-tree>

В этом разделе вы узнаете об организации исходного кода Fuchsia и инструментах управления открытым проектом.

Управление исходным кодом

Fuchsia использует инструмент `jiri` для управления git-репозиториями проекта. Он синхронизирует локальную копию исходников с манифестом Global Integration и предоставляет средства для отправки изменений обратно в Fuchsia. Global Integration — это центральный реестр, определяющий текущее состояние различных проектов в дереве Fuchsia.

Локальную копию jiri инициализируют командой import с XML-манифестом, который объявляет все репозитории и их организацию. Импорт стандартного манифеста Global Integration выглядит так:

```
jiri import -name=integration flower https://fuchsia.googlesource.com/integration
```

Эта команда добавляет манифест в локальный файл .jiri_manifest в корне вашей локальной копии.

```
<manifest>
  <imports>
    <import manifest="flower" name="integration"
      remote="https://fuchsia.googlesource.com/integration" />
  </imports>
</manifest>
```

Обратите внимание, что стандартный интеграционный манифест называется flower («цветок»). Эта метафора часто применяется к коду Fuchsia: ядро платформы Fuchsia считается стеблем (stem), а дополнительные внешние зависимости и связанные проекты — лепестками (petals).

Манифест flower — единая точка агрегации стебля и различных проектов-лепестков.

После инициализации локальной копии на вашей машине jiri в любой момент может получить последние изменения из Global Integration одной командой:

```
jiri update
```

Структура исходного кода

Fuchsia — большой открытый проект. Как и в любом крупном проекте, без карты-ориентира легко заблудиться. В этом разделе — обзор локальной копии Fuchsia с кратким описанием различных элементов, которые вам встретятся:

[Таблица структуры каталогов локальной копии — см. оригинал на fuchsia.dev.]

Исходный код платформы Fuchsia дополнительно разбивается на различные компоненты и службы, работающие на устройстве. Ниже — не полный список, но он подскажет интересные места, с которых можно начать изучение:

[Таблица компонентов и служб платформы — см. оригинал на fuchsia.dev.]

Упражнение: навигация по дереву исходников

В этом упражнении вы исследуете локальную копию дерева исходников Fuchsia с помощью доступных в окружении инструментов командной строки. Освоение этих инструментов повысит вашу продуктивность при работе с кодовой базой.

Поиск по дереву

Если вы не знаете, с чего начать, используйте утилиту fd для нечёткого (fuzzy) поиска каталогов и перехода к найденному результату.

Выполните следующую команду, чтобы запустить поиск fd по session_manager:

```
fd session_manager
```

Утилита печатает несколько возможных вариантов на выбор. Выберите вариант 2, чтобы перейти в `src/session/bin/session_manager`:

```
[1] src/session/bin/session_manager  
[2] src/session/tests/session_manager
```

Это позволяет легко находить нужный участок кода и переходить к нему. Если поиск достаточно конкретен и возвращает единственный результат, `fd` переходит туда автоматически.

Выполните поиск по `archivist` — службе диагностики Fuchsia для сбора логов, снимков состояния и событий жизненного цикла:

```
fd archivist
```

Обратите внимание: команда не вывела результатов, но ваш рабочий каталог автоматически сменился на `src/diagnostics/archivist`!

Это помогает начать, но часть вещей в дереве Fuchsia требует поиска внутри файлов.

Поиск внутри файлов исходников

Чтобы искать в дереве шаблоны внутри конкретных файлов, используйте команду `fx grep`.

Выполните в дереве поиск ссылок на пример `hello-world` с помощью `fx grep`:

```
fx grep hello-world
```

Это вернёт длинный список ссылок со всего дерева, так как пример упоминается в документации, файлах сборки и других источниках.

Поиск можно уточнить фильтрами. Выполните тот же поиск снова, но только в файлах сборки GN, используя фильтр:

```
fx grep hello-world -- build
```

Результаты показывают, что определение протокола находится в `examples/hello_world`. Эту информацию можно объединить с `fd`, чтобы перейти туда:

```
fd hello_world
```

С помощью `fx grep` найдите компоненты, реализующие FIDL-протокол `fuchsia.component.runner`. Сколько их?

Система сборки (GN/Ninja)

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/build/build-system>

Сборки Fuchsia используют Generate Ninja (GN) — мета-систему сборки, которая генерирует файлы сборки для Ninja, выполняющего собственно сборку. Система

сборки предоставляет инструменты для настройки сборки под конкретный продукт и шаблоны для сборки кода под цели Fuchsia.

Цели сборки (build targets)

Отдельные цели сборки для GN определяются в файлах BUILD.gn, расположенных рядом с исходным кодом проекта. Система сборки Fuchsia предоставляет шаблоны в виде GN-импортов (.gni) для объявления артефактов Fuchsia, таких как:

- `fuchsia_component()`: определяет исполняемый компонент, содержащий манифест, программный бинарник и ресурсы.
- `fuchsia_package()`: определяет пакет с одним или несколькими компонентами для распространения в репозитории пакетов.
- `fuchsia_test_package()`: определяет пакет с тестовыми компонентами.

Ниже — пример файла BUILD.gn для простого пакета-компонента с тестами:

```
import("//build/components.gni")

executable("bin") {
  sources = [ "main.cc" ]
}

fuchsia_component("hello-world-component") {
  deps = [ ":bin" ]
  manifest = "meta/hello-world.cml"
}

fuchsia_package("hello-world") {
  deps = [
    ":hello-world-component",
  ]
}

fuchsia_component("hello-world-test-component") {
  testonly = true
  deps = [ ":bin_test" ]
  manifest = "meta/hello-world-bin-test.cml"
}

fuchsia_test_package("hello-world-tests") {
  test_components = [ ":hello-world-test-component" ]
}
```

Всё, что участвует в сборке, идентифицируется уникальной меткой (label) из имени цели и пути к её файлу BUILD.gn. В примере выше цель `hello-world` могла бы иметь метку вида `//src/examples/basic:hello-world`.

Конфигурация сборки

Фронтенд GN настраивает сборку согласно выбранной конфигурации продукта Fuchsia, собирая все необходимые пакеты и компоненты. Эти цели определены в различных файлах BUILD.gn по всему дереву. Результат шага GN — оптимизированный набор инструкций для Ninja в каталоге сборки.

Система сборки вызывает GN, когда вы выполняете команду `fx set` для настройки сборки.

```
fx set minimal.x64
```

Шаг конфигурации GN следует выполнять каждый раз, когда нужно изменить конфигурацию продукта или набор доступных пакетов. GN также вызывается автоматически при сборке всякий раз, когда меняется один из файлов BUILD.gn текущей конфигурации.

Платы (boards) и продукты (products)

Система сборки Fuchsia определяет базовую конфигурацию сборки как сочетание продукта и платы. Вместе они образуют конфигурацию сборки, которую вы передаёте в `fx set`.

Платы определяют целевую архитектуру сборки, что может влиять на включаемые драйверы и специфичные для устройства параметры ядра.

Этот codelab нацелен на плату x64, поддерживающую эмулятор Fuchsia (FEMU) на архитектуре x64.

Продукт определяет программную конфигурацию, которую производит сборка. Она может включать набор доступных служб и пользовательский опыт.

Этот codelab нацелен на продукт `minimal_eng`.

Сборка

После завершения конфигурации GN Ninja использует сгенерированные файлы сборки и выполняет соответствующие команды компиляции, компоновки и упаковки для создания образа Fuchsia.

Система сборки вызывает Ninja, когда вы выполняете команду `fx build` для запуска текущей конфигурации сборки.

```
fx build
```

Упражнение: сборка Minimal

В этом упражнении вы соберёте конфигурацию продукта `minimal_eng` из исходников для запуска на эмуляторе платы x64.

Настройка сборки

Настройте окружение сборки для продукта `minimal` и платы x64:

```
fx set minimal.x64
```

Эта команда запускает GN на наборе целей, определённых в конфигурации сборки продукта, чтобы создать инструкции сборки. Сама сборка при этом не выполняется — задаются лишь параметры того, что считается собираемым.

Просмотр конфигурации сборки

После настройки сборки используйте `fx list-packages`, чтобы вывести набор пакетов, известных сборке:

```
fx list-packages
```

Это удобный способ проверить, был ли нужный пакет корректно включён в конфигурацию сборки.

Сборка Fuchsia Minimal

Соберите цель Minimal командой `fx build`:

```
fx build
```

Перезапуск эмулятора

- Выполните команду, чтобы закрыть открытые экземпляры эмулятора: `ffx emu stop --all`
- Запустите новый экземпляр: `ffx emu start --headless`. По завершении запуска эмулятор печатает сообщение о готовности и возвращает управление.

Осмотр устройства

Откройте другое окно терминала и выполните следующую команду, чтобы вывести детали вашего целевого устройства:

```
ffx target show
```

Найдите в выводе конфигурацию сборки цели:

```
Version: "2000-01-01T12:00:00+00:00"  
Product: "minimal_eng"  
Board: "x64"  
Commit: "2000-01-01T12:00:00+00:00"
```

Обратите внимание, что конфигурация указывает на сборку, которую вы только что выполнили на своей машине.

Теперь вы используете собственную сборку Fuchsia!

Упражнение: запуск сервера репозитория и раздача пакетов

В этом упражнении вы запустите сервер репозитория, чтобы раздать пакеты из набора `universe` на работающую цель Fuchsia в эмуляторе.

Подробнее о репозитории и разных наборах пакетов см. RFC-0212 Package Sets.

Например, команда `ffx debug connect` завершится ошибкой, потому что цели нужен компонент, экспонирующий возможность `fuchsia.debugger.Launcher`. Она доступна в пакете `debug_agent` набора `universe` для `minimal.x64`.

Список пакетов в наборе universe

Выполните `fx list-packages`, чтобы вывести пакеты набора `universe`. Для `minimal.x64` есть только один пакет:

```
fx list-packages --universe
```

Вывод показывает только пакет `debug_agent`. Он включает компонент, предоставляющий возможность `fuchsia.debugger.Launcher`. Эту возможность мы задействуем, запустив отладчик на следующих шагах.

Запуск сервера репозитория и его регистрация на цели Fuchsia

Команда `fx serve` запускает сервер репозитория и регистрирует его на цели.

```
$ fx serve
```

При успешном выполнении команды вы увидите следующий вывод:

```
2024-05-20 12:51:03 [serve] Discovery...
2024-05-20 12:51:07 [serve] Device up
2024-05-20 12:51:07 [serve] Registering devhost as update source
Serving repository '/usr/local/google/home/amituttam/fuchsia/out/minimal.x64/amber-files' to
target 'fuchsia-emulator' over address '[:,]:8083'.
```

Оставьте эту команду работать на переднем плане. Для подробного режима передайте флаг `--verbose` или `-v` в `fx serve`.

Упражнение: запуск отладчика и список процессов

С работающим сервером репозитория и целью Fuchsia, настроенной на разрешение пакетов из него, мы можем выполнить команду, которая разрешит пакет и запустит компонент внутри.

Сначала запустим просмотр логов, чтобы видеть, что происходит:

```
ffx log --filter debug_agent
```

Выполните команду выше в отдельном терминале — вывод будет пустым. Оставьте её работать. Здесь мы фильтруем по `debug_agent`, так как системе нужно разрешить этот пакет для запуска отладчика.

В другом терминале подключите отладчик к работающей системе:

```
ffx debug connect
```

Команда должна запустить `zxdb` и поместить вас в интерактивную оболочку. Если переключиться обратно в терминал с работающим `ffx log`, вы увидите вывод вроде:

```
[01949.544917][pkg-resolver] INFO: attempting to resolve fuchsia-pkg://fuchsia.com/debug_agent as
fuchsia-pkg://devhost/debug_agent with TUF
[01949.624075][pkg-resolver] INFO: updated local TUF metadata for "fuchsia-pkg://devhost" to
version RepoVersions { root: 9, timestamp: Some(1715399460), snapshot: Some(1715399460),
targets: Some(1715399460) } while getting merkle for TargetPath("debug_agent/0")
[01949.835760][pkg-resolver] INFO: resolved fuchsia-pkg://fuchsia.com/debug_agent as
fuchsia-pkg://devhost/debug_agent to
3f9783abed30d70b72d5f0730bd6e6033481073126aac0b74cbbf2d14909497e with TUF
[01949.882891][debugger] INFO: [main_launcher.cc(182)] Start listening on FIDL
fuchsia::debugger::Launcher.
```

Здесь система пытается разрешить пакет `debug_agent` и успешно делает это из настроенного репозитория `devhost`. После разрешения пакета запускается компонент, слушающий протокол запуска отладчика.

Вернувшись в `zxdb`, теперь можно выполнить `ps`, чтобы увидеть список процессов в системе.

```
[zxdb] ps
j: 1033 root
p: 1102 bin/component_manager
j: 1649
j: 1780 bootstrap/console fuchsia-boot:///console#meta/console.cm
p: 1822 console.cm
j: 1989 bootstrap/archivist fuchsia-boot:///archivist#meta/archivist.cm
p: 2051 archivist.cm
j: 2064 bootstrap/console-launcher fuchsia-boot:///console-launcher#meta/console-launcher.cm
...
```

Продукты и пакеты сборки

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/build/product-packages>

Конфигурация продукта инкапсулирует набор пакетов и компонентов, необходимых для целевого пользовательского опыта на конкретном оборудовании. Каждая конфигурация продукта объявляет стандартный список пакетов их GN-метками в файле GN-импорта (`.gni`). Содержимое текущей конфигурации сборки можно дополнять несколькими способами.

Наборы зависимостей

Пакеты в конфигурации сборки относятся к одному из трёх наборов зависимостей:

- **Base**: системно-критичные пакеты, встроенные в образ системы. Base-пакеты всегда разрешаются в версию из образа системы, даже если в репозитории доступна другая версия. Обновление base-пакета требует системного обновления (OTA).
- **Cache**: дополнительные пакеты, встроенные в образ системы. Кешированные пакеты обновляются из репозитория, если доступна другая версия.
- **Universe**: всё остальное. Этот набор доступен только через репозиторий и доставляется по требованию.

Ниже — пример фрагмента файла конфигурации продукта. Этот продукт наследует все пакеты, определённые в продукте `core`, и добавляет несколько пакетов в каждый набор зависимостей:

```
import("//products/core.gni")

legacy_base_package_labels += [
  "//src/chromium:web_engine",
  "//src/fonts",
]

legacy_cache_package_labels += [
  "//src/media/playback/bundles:services",
]

discoverable_package_labels += [
  "//src/ui/examples:bouncing_ball",
]
```

В этом примере система сборки упаковала бы среды выполнения, ресурсы и службы в образ диска целевого устройства. Примеры (examples) были бы доступны только по требованию с сервера пакетов.

Ручная настройка сборки

Создание новой полной конфигурации продукта — не единственный способ настройки сборки. Можно также передавать дополнительные метки команде `fx set` этими флагами:

- `--with`: добавить метку цели в набор `universe`.
- `--with-base`: добавить метку цели в набор `base`.
- `--with-cache`: добавить метку цели в набор `cache`.

Это удобный способ временно включить пакеты для разработки, которые не нужны в финальной сборке, например тесты. Например, следующая команда добавляет все пакеты из набора тестов Fuchsia к стандартной сборке `workstation_eng`.

```
fx set workstation_eng.x64 --with //bundles/tests
```

Команда `fx set` и эти параметры в итоге наполняют аргументы сборки, которые GN использует для генерации конфигурации. Метки пакетов — типичный пример аргументов сборки в действии.

Текущие аргументы сборки хранятся в файле `args.gn` в каталоге вывода сборки. Для более сложных сценариев можно использовать `fx args`, чтобы интерактивно отредактировать `args.gn` перед генерацией конфигурации.

Полный список поддерживаемых аргументов сборки см. в «GN Build Arguments».

Разработка с пакетами

Вспомните из введения, что пакеты Fuchsia не «устанавливаются» на устройство, а разрешаются из локального кеша пакетов или репозитория. При разработке ПО для Fuchsia это значит, что тестирование вашего кода включает следующие шаги:

- Собрать обновлённый пакет с изменениями кода.
- Опубликовать обновлённые пакеты в репозиторий.
- Инициировать обновление на целевом устройстве.

Инструменты разработчика, такие как `fx build`, публикуют обновления пакетов в локальный репозиторий в процессе сборки. Набор зависимостей, в котором продукт объявляет пакет, определяет, как инициировать обновление на устройстве:

- Base-пакеты можно обновить только полным обновлением системы (OTA). Запустите системное обновление командой `fx ota` или прошейте новый образ системы на устройство.
- Cache- и universe-пакеты обновляются автоматически при следующем разрешении пакета.

Упражнение: настройка сборки

В этом упражнении вы настроите сборку `workstation_eng`, временно включив дополнительные пакеты в набор `universe`, чтобы сделать их доступными целевому устройству.

Добавление пакетов в сборку

Дополнительные цели можно включить в конфигурацию сборки флагом `--with` команды `fx set`. Перенастройте сборку `workstation_eng`, включив все примеры Fuchsia:

```
fx set workstation_eng.x64 --with //examples
```

Так обычно включают тестовые пакеты, которые нужно запустить на устройстве, или новый пакет, над которым вы работаете и который ещё не входит в конфигурацию продукта.

Проверьте, что пакеты примеров добавлены в сборку:

```
fx list-packages example
```

Какой набор пакетов использовался для включения примеров? Сможете ли вы подтвердить это фильтрами, поддерживаемыми `list-packages`?

Как выглядят аргументы сборки при выполнении `fx args`?

Изучение целей сборки с помощью GN

В GN есть мощный набор диагностических инструментов для изучения целей вашей сборки. Далее — как использовать подкоманды `fx gn` для исследования целей.

Начните с команды `desc`, чтобы вывести детали о наборе `//examples`, который вы только что добавили в сборку.

```
fx gn desc out/default //examples
```

Команда печатает детали о типе цели и всех её зависимостях. Для набора `//examples` список зависимостей представляет отдельные пакеты примеров, добавленные в сборку.

Изучите детали цели `hello-world` той же командой.

```
fx gn desc out/default //examples/hello_world:hello-world
```

Эта цель представляет пакет с несколькими компонентами, поэтому вы увидите дополнительные зависимости, такие как манифесты компонентов и метаданные пакета.

Спуститесь ещё на уровень, чтобы изучить компонент `hello-world-rust`.

```
fx gn desc out/default //examples/hello_world/rust:hello-world-rust-component
```

С точки зрения конкретной цели, например компонента `hello-world-rust`, можно посмотреть и вверх по графу сборки командой `refs`. Она показывает входящие ссылки на заданную цель.

```
fx gn refs out/default //examples/hello_world/rust:hello-world-rust-component
```

Наконец, командой `path` можно показать цепочку ссылок между любыми двумя целями. Это помогает понять, где и как ваша цель вообще включена в сборку, проверив путь ссылок между ней и `//:default`.

```
fx gn path out/default //:default //examples/hello_world:hello-world
```

Запуск примера в эмуляторе

Выполните `fx build` снова, чтобы собрать обновлённые пакеты:

```
fx build
```

Командой `ffx component` запустите пример компонента Hello World:

```
ffx component run /core/ffx-laboratory:hello-world-cpp  
fuchsia-pkg://fuchsia.com/hello-world-cpp#meta/hello-world-cpp.cm
```

Откройте новое окно терминала и отфильтруйте логи устройства по сообщениям из примера:

```
ffx log --filter hello
```

В логах устройства вы должны увидеть следующий вывод:

```
[hello-world-cpp][I] Hello, World!
```

Упражнение: создание нового продукта сборки

В следующем упражнении вы инкапсулируете эти дополнительные пакеты в новую конфигурацию продукта, расширяющую `workstation_eng`.

Объявление конфигурации продукта

Создайте новый каталог под `//vendor` с именем `fuchsia-codelab`:

```
mkdir -p vendor/fuchsia-codelab
```

Создайте новый файл `fuchsiab.gni` под `//vendor/fuchsia-codelab/products` со следующим содержимым:

```
# Extend the workstation_eng product  
import("//products/workstation_eng.gni")
```

Так создаётся новый продукт `fuchsiab`, расширяющий `workstation_eng` и наследующий все определённые им метки пакетов.

Проверьте, что система сборки распознаёт новый продукт, командой `fx list-products`. Вы должны увидеть `fuchsiab` в списке продуктов.

```
fx list-products
```

Добавление пакетов в ваш продукт

Чтобы настроить конкретные пакеты вашего продукта, добавьте их в метки соответствующего набора пакетов в конфигурации продукта.

Добавьте следующие строки в `//vendor/fuchsia-codelab/products/fuchsialab.gni`, чтобы включить пример Hello World в ваш продукт:

```
discoverable_package_labels += [  
  "//examples/hello_world",  
]
```

Сборка нового продукта

Перенастройте сборку для продукта `fuchsialab` на плате FEMU:

```
fx set fuchsialab.x64
```

Проверьте, что пакет примера Hello World теперь часть сборки:

```
fx list-packages hello
```

Выполните `fx build`, чтобы сгенерировать новый образ вашего продукта:

```
fx build
```

Перезапуск эмулятора

- Выполните команду, чтобы закрыть открытые экземпляры эмулятора: `ffx emu stop --all`
- Запустите новый экземпляр: `ffx emu start --headless`. По завершении запуска эмулятор печатает сообщение о готовности и возвращает управление.

Осмотр устройства

Откройте другое окно терминала и выполните следующую команду, чтобы вывести детали вашего целевого устройства:

```
ffx target show
```

Найдите в выводе конфигурацию сборки цели:

```
Version: "2000-01-01T12:00:00+00:00"  
Product: "fuchsialab"  
Board: "x64"  
Commit: "2000-01-01T12:00:00+00:00"
```

Поздравляем! Вы только что собрали собственный продукт на основе Fuchsia!

Работа с устройствами

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/build/working-with-devices>

Эта серия codelab сосредоточена на эмуляторе Fuchsia (FEMU) как целевом устройстве: он собирается и распространяется вместе с деревом исходников и работает на вашей машине. Однако Fuchsia можно собрать и для поддерживаемых

аппаратных платформ, например Intel NUC.

В этом разделе описаны некоторые особенности работы с Fuchsia на физических устройствах.

Настройка

Fuchsia определяет поддержку аппаратных устройств именем платы (board), используемым для настройки сборки. Сюда входят специфичные для оборудования пакеты, например драйверы. Вспомните ранее использованную команду `fx set`:

```
fx set workbench_eng.x64
```

Здесь `x64` — имя платы, которая может работать на FEMU и Intel NUC. Чтобы собрать тот же продукт для Khadas VIM3, измените команду `set` на плату `vim3`.

```
fx set workbench_eng.vim3
```

Теперь выполнение `fx build` сгенерирует образ для целевого устройства.

Bootstrap (начальная подготовка)

Перед прошивкой ОС на поддерживаемом устройстве должен быть установлен совместимый с Fuchsia загрузчик. Этот процесс называется *bootstrapping* устройства. На многих устройствах совместимый загрузчик установлен с завода; для других могут потребоваться инструменты производителя для обновления загрузчика до совместимой версии. Подробнее см. документацию по вашему устройству.

Flash (прошивка)

Процесс загрузки ОС на устройство называется прошивкой (*flashing*). С устройством в режиме загрузчика, подключённым к вашему рабочему месту, можно прошить Fuchsia командой `flash`.

```
fx flash
```

Уже прошитые устройства можно перезагрузить из Fuchsia в режим загрузчика, если их нужно прошить снова, с помощью `ffx`:

```
ffx target reboot --bootloader
```

Discover (обнаружение)

Обнаруживать устройства Fuchsia и взаимодействовать с ними можно с рабочей машины, подключённой по USB или локальной сети IPv6. Fuchsia обеспечивает автоматическое обнаружение устройств через DNS Service Discovery (DNS-SD) поверх multicast DNS (mDNS) и mesh-протокол Overnet.

Хост-инструменты, такие как `ffx`, обнаруживают анонсирующие себя устройства и обеспечивают взаимодействие хоста с целью — как с физическими устройствами, так и с FEMU.

```
ffx target list
```

NAME	SERIAL	TYPE	STATE	ADDRS/IP	RCS
fuchsia-5254-0063-5e7a	<unknown>	.	Product	[fe80::c357:53e7:aedf:ed95%qemu]	Y

Если целевое устройство не анонсирует пакеты обнаружения или ffx не может их обнаружить, такими целями можно управлять вручную командами add и remove:

```
ffx target add device-ip:device-port  
ffx target remove device-ip:device-port
```

Как только устройство отслеживается в списке целей, ffx взаимодействует со службой Remote Control Service (RCS) на цели, что позволяет отправлять дополнительные команды.

Что дальше?

Поздравляем! Вы успешно настроили и собрали Fuchsia из исходников и теперь лучше понимаете, где в дереве находятся ключевые системные компоненты.

В следующем модуле вы узнаете больше о сборке фундаментальной единицы ПО Fuchsia:

Компоненты Fuchsia

Часть III. Компоненты

Компоненты: обзор

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/components>

Компонент — это программа, выполняющаяся в Fuchsia в собственной песочнице. Каждый компонент — это компонуемый модуль, взаимодействующий с другими компонентами через их возможности. Всё ПО в Fuchsia определено как компонент, кроме образа ядра, процессов начальной загрузки и менеджера компонентов.

Фреймворк компонентов Fuchsia отвечает за выполнение почти всего ПО пользовательского пространства в системе. Менеджер компонентов — системный процесс, который координирует выполнение и жизненный цикл всех экземпляров компонентов, поддерживает топологию компонентов, предоставляет им возможности и изолирует их друг от друга.

Компоненты получают привилегии на доступ к различным частям системы через возможности. Каждый компонент может объявлять новые возможности, предлагаемые системе, и возможности других компонентов (или фреймворка), необходимые ему для работы. Менеджер компонентов разрешает и проверяет все запросы возможностей между компонентами, гарантируя их соответствие тем возможностям, которые были корректно предоставлены компоненту.

Предварительные требования

- Введение в Fuchsia
- Сборка Fuchsia

Что вы узнаете

- Принципы проектирования компонентов Fuchsia
- Объявление компонентов и возможностей
- Тестирование и отладку компонента

Что вам понадобится

- Рабочая машина под управлением Linux
- Эмулятор Fuchsia (FEMU)
- Успешно собранная Fuchsia с конфигурацией `fx set workstation_eng.x64`

Объявление компонентов

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/components/declaring-components>

У каждого компонента есть объявление (declaration), описывающее его атрибуты и возможности. Для компонентов, распространяемых в пакетах, объявление выражается файлом манифеста компонента и загружается с помощью разрешителя компонентов (component resolver).

Компоненты объявляют на языке манифеста компонентов (CML). На этапе сборки инструмент Component Manifest Compiler (cmc) проверяет и компилирует исходный

манифест в двоичный формат (.cm) и сохраняет его в пакете компонента. На этапе выполнения разрешители компонентов загружают двоичный манифест в FIDL-структуру ComponentDecl для менеджера компонентов.

Манифесты компонентов

CML-файлы — это файлы JSON5 с расширением .cm. Ниже — пример CML-манифеста для простого компонента, запускающего ELF-бинарник, который печатает сообщение «Hello, World» в системный лог:

```
{
  // Information about the program to run.
  program: {
    // Use the built-in ELF runner.
    runner: "elf",
    // The binary to run for this component.
    binary: "bin/hello",
    // Program arguments
    args: [
      "Hello",
      "World!",
    ],
  },

  // Capabilities used by this component.
  use: [
    { protocol: "fuchsia.logger.LogSink" },
  ],
}
```

Этот файл объявляет два основных раздела информации о компоненте:

- **program:** описывает сведения об исполняемом файле — бинарник, аргументы программы и связанную среду выполнения. В этом примере бинарник скомпилирован как ELF-исполняемый файл и использует встроенный ELF-runner.
- **use:** объявляет возможности, необходимые компоненту для работы. В этом примере протокол `fuchsia.logger.LogSink` позволяет компоненту писать сообщения в системный лог (syslog).

Шарды манифеста (manifest shards)

Некоторые наборы возможностей представляют требования сценариев, общие для многих компонентов системы, например логирование. Чтобы упростить включение таких возможностей, фреймворк абстрагирует их в шарды манифеста, которые можно включать в ваш CML-файл.

Ниже — CML, эквивалентный предыдущему примеру. Здесь необходимые возможности логирования предоставляются включением `diagnostics/syslog/client.shard.cm` вместо явного объявления `fuchsia.logger.LogSink`:


```
{
  include: [ "syslog/client.shard.cml" ],

  // Information about the program to run.
  program: {
    // Use the built-in ELF runner.
    runner: "elf",
    // The binary to run for this component.
    binary: "bin/hello-world",
    // Program arguments
    args: [
      "Hello",
      "World!",
    ],
  },
}
```

Сборка компонентов

Система сборки Fuchsia предоставляет шаблоны как GN-импорты в `//build/components.gni` для сборки и упаковки ПО в компоненты Fuchsia. Ниже — пример файла BUILD.gn для простого компонента на C++:

```
import("//build/components.gni")

executable("bin") {
  sources = [ "main.cc" ]
}

resource("my_file") {
  sources = [ "my_file.txt" ]
  outputs = [ "data/{{source_file_part}}" ]
}

fuchsia_component("hello-world-component") {
  component_name = "hello-world"
  deps = [
    ":bin",
    ":my_file",
  ]
  manifest = "meta/hello-world.cml"
}

fuchsia_package("hello-world") {
  package_name = "hello-world"
  deps = [
    ":hello-world-component",
  ]
}
```

Этот файл содержит следующие основные элементы:

- `executable()`: компилирует исходный код в бинарник. Эта цель зависит от языка программирования. Например, `executable` — для C++, `rustc_binary` — для Rust, `go_binary` — для Golang.
- `resource()`: необязательный именованный набор файлов данных, копируемых как ресурсы в другую GN-цель. Эти файлы доступны бинарнику внутри пространства имён компонента.
- `fuchsia_component()`: собирает воедино бинарник, манифест компонента и дополнительные ресурсы. Эта цель компилирует исходный манифест в

объявление компонента с помощью `src`.

- `fuchsia_package()`: единица распространения компонентов. Позволяет разместить один или несколько компонентов в репозитории пакетов и включить их в наборы пакетов целевого устройства. Эта цель генерирует метаданные пакета и собирает файл Fuchsia Archive (`.far`).

Пакеты могут содержать несколько компонентов, перечисленных как `deps` в шаблоне `fuchsia_package()`. Файл сборки для пакета с единственным компонентом можно упростить шаблоном `fuchsia_package_with_single_component()`.

Следующий упрощённый пример `BUILD.gn` эквивалентен предыдущему:

```
import("//build/components.gni")

executable("bin") {
  sources = [ "main.cc" ]
}

resource("my_file") {
  sources = [ "my_file.txt" ]
  outputs = [ "data/{{source_file_part}}" ]
}

fuchsia_package_with_single_component("hello-world") {
  manifest = "meta/hello-world.cml"
  deps = [
    ":bin",
    ":my_file",
  ]
}
```

Упражнение: создание нового компонента

В этом упражнении вы соберёте и запустите базовый компонент, который читает аргументы программы и выводит приветствие в системный лог.

Для начала создайте каркас проекта для нового компонента `echo-args` в каталоге `//vendor/fuchsia-codelab`:

```
mkdir -p vendor/fuchsia-codelab/echo-args
```

Создайте следующую структуру файлов и каталогов в каталоге нового проекта:

Добавление аргументов программы

Файл манифеста компонента определяет атрибуты исполняемого файла, включая аргументы программы, и возможности компонента. Добавьте следующее содержимое в `meta/echo.cml`:

Логирование аргументов

Откройте исходный файл главного исполняемого файла и добавьте следующие операторы импорта:

Добавьте следующий код для реализации функции `main()`:

Этот код читает аргументы программы и передаёт их функции `greeting()` для формирования ответа в записи `syslog`.

Добавьте следующий код для реализации функции `greeting()`:

Эта функция формирует простую строку из списка переданных аргументов в зависимости от его длины.

В Fuchsia два основных буфера логов: системный лог (`syslog`) и отладочный лог ядра (`klog`). По умолчанию для записи сообщений из вашего кода компонентам доступны лишь потоковые дескрипторы `stdout` и `stderr`, если у них есть доступ к системному логу. В Fuchsia также есть библиотеки логирования для большего контроля над метаданными и форматом сообщений.

Подробнее о логировании из кода см. «Recording Logs».

Добавление в конфигурацию сборки

Обновите зависимости программы в файле `BUILD.gn`:

Добавьте новый компонент в конфигурацию сборки:

```
fx set workstation_eng.x64 --with //vendor/fuchsia-codelab/echo-args
```

Выполните `fx build` и убедитесь, что сборка завершается успешно:

```
fx build
```

В следующем разделе вы интегрируете этот компонент в сборку и протестируете вывод в системном логе.

Организация компонентов

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/components/organizing-components>

Все компоненты системы образуют дерево экземпляров компонентов с общим корнем. Родительские компоненты отвечают за объявление экземпляров других компонентов как своих детей и предоставление им возможностей. Дочерние компоненты, в свою очередь, могут экспонировать возможности обратно родителю. Эти отношения экземпляров и возможностей образуют топологию компонентов.

Любой родительский компонент со всеми детьми образует группу в дереве — `realm`. Realm позволяет родителю контролировать, какие возможности входят в его поддереву и выходят из него, создавая границу возможностей. Такая инкапсуляция позволяет реорганизовывать `realm` внутри, не затрагивая внешние компоненты, зависящие от его экспонированных возможностей.

На схеме выше возможность-протокол `fuchsia.example.Foo` маршрутизируется через дерево экземпляров от поставщика к клиенту. Компоненты объявляют требуемые возможности ключевым словом `use`:

```

{
  // Information about the program to run.
  program: {
    // Use the built-in ELF runner to run core binaries.
    runner: "elf",
    // The binary to run for this component.
    binary: "bin/client",
  },

  // Capabilities required by this component.
  use: [
    { protocol: "fuchsia.example.Foo" },
  ],
}

```

Компоненты объявляют возможности, которые они реализуют (предоставляют), в разделе `capabilities` манифеста. Так возможность и её поставщик становятся известны фреймворку. См. пример `provider.cml`:

```

{
  // Information about the program to run.
  program: {
    // Use the built-in ELF runner to run core binaries.
    runner: "elf",
    // The binary to run for this component.
    binary: "bin/provider",
  },

  // Capabilities provided by this component.
  capabilities: [
    { protocol: "fuchsia.example.Foo" },
  ],
  // Capabilities routed through this component.
  expose: [
    {
      protocol: "fuchsia.example.Foo",
      from: "self",
    },
  ],
}

```

Ключевое слово `expose` делает возможность доступной из этого компонента другим `realm` через его родителя; сюда могут входить и возможности, предоставляемые детьми этого компонента. В данном случае источник возможности — `self`, поскольку поставщиком является сам компонент.

Родительские компоненты управляют маршрутизацией возможностей внутри `realm`, создавая явные пути от клиента к поставщику. См. пример манифеста `parent.cml`:

```
{
  children: [
    {
      name: "provider",
      url: "fuchsia-pkg://fuchsia.com/foo-package#meta/provider.cm",
    },
    {
      name: "client",
      url: "fuchsia-pkg://fuchsia.com/foo-package#meta/client.cm",
    },
  ],
  offer: [
    {
      protocol: "fuchsia.example.Foo",
      from: "#provider",
      to: [ "#client" ],
    },
  ],
}

```

Родительский компонент объявляет набор дочерних компонентов в `realm` и маршрутизирует к ним возможности ключевым словом `offer`. Так родитель определяет и область, и источник возможностей каждого ребёнка. Это также позволяет нескольким компонентам в топологии предоставлять одну и ту же возможность, поскольку фреймворк опирается на явные маршруты, чтобы понять, как разрешать запросы каждого клиента.

Типы возможностей

Компоненты Fuchsia поддерживают много разных типов возможностей. До сих пор в этом модуле примеры демонстрировали два типа: `runner` и `protocol`. Возможно, вы заметили, что возможность `protocol` требует пути маршрутизации, а `runner` — нет.

Некоторые возможности явно маршрутизируются компонентам их родителем, а другие предоставляются всем компонентам одного `realm` через окружения (`environments`). Окружения позволяют фреймворку выделять возможности, которые не имеет смысла маршрутизировать явно для каждого компонента. По умолчанию компонент наследует окружение родителя. Компоненты также могут объявлять новое окружение для своих детей.

Один из принципов безопасности Fuchsia — «нет эмбиентных полномочий» (no ambient authority) для программ в системе. Это значит, что каждая операция должна быть привязана к объектной возможности, а не предоставляться на основе более высокоуровневого контекста, такого как личность пользователя или роль.

Фреймворк компонентов поддерживает этот принцип, гарантируя, что компоненты имеют прямой доступ только к возможностям, явно маршрутизированным их родителем. Доступ к возможностям, выделяемым через окружения, опосредуется менеджером компонентов от имени компонента.

В следующей таблице перечислены типы возможностей, доступные компонентам, и указано, должны ли они явно маршрутизироваться от родителя или предоставляются окружением:

[Таблица типов возможностей — см. оригинал на fuchsia.dev.]

Идентификация компонентов

Компоненты идентифицируются URL. Фреймворк разрешает URL компонента в его объявление с помощью разрешителя компонентов (component resolver). Разрешители сами являются компонентами, способными обрабатывать определённую схему URL и получать манифест, программу и ресурсы компонента.

Большинство компонентов публикуются внутри пакета Fuchsia, поэтому URL компонента — это ссылка на манифест компонента внутри этого пакета. См. пример:

```
fuchsia-pkg://fuchsia.com/foo-package#meta/foo-component.cm
```

Экземпляры компонентов идентифицируются ссылкой на топологический путь, называемой moniker. Moniker указывает расположение компонента в дереве экземпляров как абсолютный или относительный путь. Например, moniker /core/system-updater ссылается на экземпляр system-updater в realm core.

Жизненный цикл компонента

Экземпляры компонентов создаются и уничтожаются при добавлении и удалении в топологии. Это происходит одним из двух способов:

- Статически: экземпляр объявлен в манифесте компонента как ребёнок другого компонента в дереве. Статические компоненты создаются и уничтожаются только при обновлении, меняющем топологию.
- Динамически: экземпляр добавляется или удаляется в коллекции компонентов во время выполнения через протокол fuchsia.component.Realm. Динамические компоненты уничтожаются при выключении системы.

После уничтожения компонента фреймворк удаляет его постоянное состояние (например, локальное хранилище).

Фреймворк запускает экземпляр компонента, когда другой компонент пытается открыть к нему канал. Это происходит неявно при подключении к возможности, экспонированной компонентом. Подключение к уже запущенному компоненту повторно использует работающий экземпляр.

Компоненты могут останавливаться сами, завершая программу (как определено runner компонента), либо фреймворк может остановить компонент в рамках выключения системы.

Упражнение: интеграция компонентов

Чтобы компонент можно было вызвать, он должен присутствовать в активной топологии. В этом упражнении вы добавите компонент в ffx-laboratory — ограниченную коллекцию для разработки внутри realm core продукта. Коллекции позволяют динамически создавать и уничтожать компоненты во время выполнения.

Запуск эмулятора

Если у вас ещё нет запущенного экземпляра, запустите эмулятор:

- Запустите новый экземпляр эмулятора: `ffx emu start --headless`. По завершении запуска эмулятор печатает сообщение о готовности и возвращает управление.
- Запустите сервер пакетов, чтобы эмулятор мог загружать программные пакеты:
`fx serve`

Добавление в топологию компонентов

Создайте новый экземпляр компонента `echo-args` следующей командой:

```
ffx component create /core/ffx-laboratory:echo-args \  
fuchsia-pkg://fuchsia.com/echo-args#meta/echo-args.cm
```

Эта команда принимает два параметра:

- `/core/ffx-laboratory:echo-args`: это `moniker` компонента, представляющий путь экземпляра в топологии.
- `fuchsia-pkg://fuchsia.com/echo-args#meta/echo_args.cm`: это URL компонента, указывающий, как Fuchsia должна разрешить компонент из сервера пакетов.

Теперь в топологии существует новый экземпляр `echo-args`. Покажите его детали следующей командой:

```
ffx component show echo-args
```

Вы должны увидеть следующий вывод:

```
Moniker: /core/ffx-laboratory:echo-args  
URL: fuchsia-pkg://fuchsia.com/echo-args#meta/echo-args.cm  
Type: CML dynamic component  
Component State: Unresolved  
Execution State: Stopped
```

Обратите внимание: экземпляр создан, но URL компонента ещё не разрешён. Разрешение происходит, когда фреймворк пытается запустить экземпляр.

Запуск экземпляра компонента

Запустите новый экземпляр `echo-args` следующей командой:

```
ffx component start /core/ffx-laboratory:echo-args
```

Эта команда принимает один параметр:

- `/core/ffx-laboratory:echo-args`: это `moniker` компонента, представляющий путь экземпляра в топологии.

Это запускает экземпляр компонента, печатает приветствие в лог и завершается. Откройте новое окно терминала и отфильтруйте логи устройства по сообщениям из примера:

```
ffx log --filter echo
```

В логах устройства вы должны увидеть следующий вывод:

```
[ffx-laboratory:echo-args][I] Hello, Alice, Bob, Spot!
```

Исследование экземпляра

Снова покажите детали экземпляра echo-args следующей командой:

```
ffx component show echo-args
```

Теперь вы должны увидеть следующий вывод:

```
Moniker: core/ffx-laboratory:echo-args
URL: fuchsia-pkg://fuchsia.com/echo-args#meta/echo-args.cm
Type: CML dynamic component
Component State: Resolved
Incoming Capabilities: fuchsia.logger.LogSink
Exposed Capabilities: diagnostics
Execution State: Stopped
```

Состояние компонента изменилось на Resolved, и вы видите больше деталей о его возможностях.

У компонентов нет эмбиентных возможностей для доступа к другим частям системы. Каждая нужная возможность должна быть явно маршрутизирована компоненту через топологию или предоставлена его окружением.

Компоненту echo-args нужна возможность fuchsia.logger.LogSink для записи в системный лог. Вы смогли увидеть вывод лога, потому что эта возможность предлагается компонентам коллекции ffx-laboratory из realm core:

```
{
  collections: [
    {
      name: "ffx-laboratory",
    },
  ],
  offer: [
    {
      protocol: [ "fuchsia.logger.LogSink" ],
      from: "parent",
      to: "#ffx-laboratory",
    },
  ],
}
```

Необходимые возможности логирования в проекте неявно объявлены шардом манифеста syslog: syslog/client.shard.cml.

Уничтожение экземпляра

Очистите экземпляр echo-args следующей командой:

```
ffx component destroy /core/ffx-laboratory:echo-args
```

Тестирование компонентов

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/components/component-tests>

Фреймворк запуска тестов Fuchsia (Test Runner Framework) позволяет разработчикам писать тесты для компонентов на разных языках и средах выполнения и выполнять их на целевом устройстве. Фреймворк предоставляет компоненты-раннеры тестов,

реализующие протокол `fuchsia.test.Suite` и интегрирующиеся с распространёнными языковыми фреймворками тестирования, такими как `GoogleTest` (C++).

Компонент `test_manager` отвечает за запуск тестов на устройстве Fuchsia. Он находит компоненты, реализующие протокол набора тестов, и запускает их как дочерние компоненты. Это значит, что `test_manager` также отвечает за предоставление возможностей каждому набору тестов, создавая так называемый `realm` тестов.

Инструменты разработчика, такие как `ffx test`, взаимодействуют с `test_manager` на устройстве для выполнения наборов тестов и получения результатов.

Раннеры тестов (test runners)

Раннеры тестов — это переиспользуемые адаптеры между Test Runner Framework и распространёнными фреймворками, на которых разработчики пишут тесты на предпочитаемом языке. Каждый компонент-раннер экспонирует возможность `fuchsia.test.Suite`, позволяющую `test_manager` перечислять и выполнять отдельные тесты, и объявляет подходящий `runner` с поддержкой фреймворка тестирования.

Для упрощения интеграции Test Runner Framework предоставляет шарды манифеста для каждого языкового раннера. Ниже — эквивалентный CML раннера тестов для объявления возможностей из предыдущего примера тестов компонента.

Модульные тесты (unit tests)

Модульное тестирование сосредоточено на проверке отдельных единиц кода внутри вашего компонента в изоляции от других компонентов системы. Модульные тесты должны быть герметичными (*hermetic*): они не требуют и не предоставляют дополнительных возможностей вне теста.

Система сборки Fuchsia предоставляет дополнительные GN-цели для упрощения модульного тестирования компонентов:

- `fuchsia_unittest_package()`: однокомпонентный пакет, который автоматически генерирует минимальный манифест для ссылки на тестовый бинарник и не требует дополнительных возможностей.
- `fuchsia_unittest_component()`: объявление компонента, генерирующее такой же минимальный манифест. Это правило полезно, если нужно собрать несколько компонентов модульных тестов в один `fuchsia_package()`.

Ниже — пример фрагмента `BUILD.gn` для включения модульных тестов:

```
import("//build/components.gni")

executable("bin_test") {
  sources = [ "main_test.cc" ]
  deps = [
    "//src/lib/fxl/test:gtest_main",
    "//third_party/googletest:gtest",
  ]
  testonly = true
}

fuchsia_unittest_package("hello-world-unittests") {
  deps = [
    ":bin_test",
  ]
}
```

Упражнение: модульные тесты echo

В этом упражнении вы добавите модульные тесты к компоненту echo-args с помощью Test Runner Framework и запустите их в окружении FEMU.

Реализация модульных тестов

Модульные тесты проверяют, что внутренние функции компонента работают как ожидается. Для echo-args вы проверите, что функция greeting() из предыдущего упражнения возвращает ожидаемые значения.

Добавьте следующие функции модульных тестов, чтобы проверить поведение greeting() при одном, двух и трёх аргументах:

Обновление конфигурации сборки

Добавьте следующие правила в файл BUILD.gn, чтобы сгенерировать новый пакет модульных тестов:

Это правило упаковывает ваши модульные тесты в компонент со следующим URL:

```
fuchsia-pkg://fuchsia.com/echo-args-unittests#meta/echo-args-unittests.cm
```

Запуск модульных тестов

Обновите верхнеуровневую цель сборки, чтобы собирать и пакет компонента, и пакет тестов:

Выполните `fx build` снова, чтобы собрать пакет тестов:

```
fx build
```

Командой `ffx test` выполните модульные тесты внутри этого пакета. Убедитесь, что тесты проходят:

```
ffx test run \
  fuchsia-pkg://fuchsia.com/echo-args-unittests#meta/echo-args-unittests.cm
```

Отладка компонентов

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/components/debugging-components>

Разработка ПО означает работу с падениями программ и поиском источников багов. В Fuchsia есть набор инструментов для выявления и диагностики проблем в компонентах — от анализа логов падений до пошаговой отладки в коде.

Анализ падений

При загрузке Fuchsia запускает программу `crashanalyzer`, которая сообщает о падениях программ и печатает трассировку (`backtrace`) упавшего потока в системный лог. Хотя их можно изучать напрямую в логах во время выполнения, содержимое трассировки кодируется ссылками на адреса в стековой памяти, а не указывает на соответствующие строки в исходных файлах программы.

```
[klog][I] devmgr: crash_analyzer_listener: analyzing exception type 0x108
[klog][I] <== fatal exception: process crasher[42410] thread initial-thread[42424]
[klog][I] <== fatal page fault, PC at 0x1e1888dbbbd7
[klog][I] CS: 0 RIP: 0x1e1888dbbbd7 EFL: 0x10246 CR2: 0
[klog][I] RAX: 0 RBX: 0x1 RCX: 0x721ad98697c6 RDX: 0x77accb36f264
[klog][I] RSI: 0 RDI: 0 RBP: 0x2781c4816f90 RSP: 0x2781c4816f80
[klog][I] R8: 0 R9: 0 R10: 0 R11: 0x246
[klog][I] R12: 0x773bf11dcda0 R13: 0x773bf11dcdd0 R14: 0x16 R15: 0x78050d69
[klog][I] errc: 0x6
[klog][I] bottom of user stack:
[klog][I] 0x00002781c4816f80: f11dcda0 0000773b 9ccd2b38 000039b2 |....;w..8+...9..|
[klog][I] 0x00002781c4816f90: c4816fd0 00002781 88dbbba7 00001e18 |.o...'.....|
[klog][I] 0x00002781c4816fa0: 00000008 00000000 9ccd2b38 000039b2 |.....8+...9..|
[klog][I] 0x00002781c4816fb0: f11dcf70 0000773b f11dcf70 0000773b |p...;w..p...;w..|
[klog][I] 0x00002781c4816fc0: cb36f570 000077ac f11dcdd0 0000773b |p.6..w.....;w..|
[klog][I] 0x00002781c4816fd0: c4816ff0 00002781 cb2b0d0f 000077ac |.o...'....+.w..|
[klog][I] 0x00002781c4816fe0: 00000054 00000000 f11dcf70 0000773b |T.....p...;w..|
[klog][I] 0x00002781c4816ff0: f11dcfe0 0000773b 00000000 00000000 |....;w.....|
[klog][I] arch: x86_64
[klog][I] dso: id=a94c78564173530d51670b6586b1aa471e004f06 base=0x7d3506a49000 name=libfdio.so
[klog][I] dso: id=a61961ba9776a67a00fb322af9ebbdcd1ce3f62 base=0x77accb297000 name=libc.so
[klog][I] dso: id=760f1e6e47d3dd8b6a19150aa47241279ec75a9c base=0x721ad9863000 name=<vDSO>
[klog][I] dso: id=b18462140c6784a53736105bbf3021852eeda68c base=0x1e1888dbb000 name=app:crasher
[klog][I] bt#01: pc 0x1e1888dbbbd7 sp 0x2781c4816f80 (app:crasher,0xbd7)
[klog][I] bt#02: pc 0x1e1888dbbba7 sp 0x2781c4816fa0 (app:crasher,0xba7)
[klog][I] bt#03: pc 0x77accb2b0d0f sp 0x2781c4816fe0 (libc.so,0x19d0f)
[klog][I] bt#04: pc 0 sp 0x2781c4817000
[klog][I] bt#05: end
```

Это потому, что по умолчанию при сборке отладочные символы удаляются из базовых бинарников. Чтобы корректно проанализировать лог падения, нужно заново применить эти символы к трассировке и увидеть стек вызовов в терминах номеров строк исходного кода. Когда вы вызываете команду `ffx log`, инструменты разработчика прогоняют сырой лог через дополнительный бинарник `symbolizer`, который заново применяет символы из вашей локальной конфигурации сборки к любым трассировкам в логе.

```
ffx log
```

В выводе вы увидите символы, заново применённые к трассировке:

```
[klog][I] devmgr: crash_analyzer_listener: analyzing exception type 0x108
... same output as "raw" backtrace ...
start of symbolized stack:
[klog][I] #01: blind_write at ../../src/developer/forensics/crasher/cpp/crasher.c:21
[klog][I] #02: main at ../../src/developer/forensics/crasher/cpp/crasher.c:137
[klog][I] #03: start_main at ../../zircon/third_party/ulib/musl/src/env/__libc_start_main.c:49
[klog][I] #04: unknown, can't find pc, sp or app/library in line
end of symbolized stack
```

С корректно символизированной трассировкой можно напрямую найти место падения в исходном коде.

Пошаговая отладка

Знания о том, где упала программа, может быть недостаточно для полной диагностики. Иногда нужно пройти по коду шаг за шагом и даже исследовать состояние переменных в памяти. Для этого в Fuchsia есть отладчик базового кода `zxdb`.

Инструмент `zxdb` — это клиент, подключающийся к работающему компоненту `debug_agent` на целевом устройстве. Командами `zxdb` можно настроить `debug_agent` на подключение к конкретным процессам и установку точек останова. После подключения сессии отладки к работающему процессу `zxdb` позволяет шагать по коду и исследовать кадры стека.

Настройка сессии отладки включает следующие высокоуровневые шаги:

- Запустить компонент `debug_agent` на целевом устройстве.
- Запустить клиент `zxdb` и подключиться к целевому устройству.
- Указать `zxdb` расположение отладочных символов.

Простейший способ начать сессию отладки — команда `ffx debug connect`, которая делает всё это в контексте вашей локальной сборки Fuchsia. Однако эти шаги можно выполнить и вручную, если нужно настроить их по отдельности.

После активации сессии отладки вы попадаете в приглашение `[zxdb]` для ввода команд отладчика. С помощью `zxdb` можно настроить `debug_agent` на подключение к процессу по фильтру имени и установить отложенные точки останова, даже если подходящий процесс ещё не запущен.

Следующий пример ставит отложенную точку останова на `main`, чтобы остановиться в начале выполнения, и ждёт запуска процесса с именем «hello-world»:

```
[zxdb] attach hello-world
Waiting for process matching "hello-world"

[zxdb] break main
Breakpoint 1 (Software) on Global, Enabled, stop=All, @ main
Pending: No matches for location, it will be pending library loads.
```

Отладчик Fuchsia поддерживает два основных способа отладки процесса: `run` и `attach`. Избегайте использования `run` для запуска и отладки компонентов.

Когда отладчик запускает компонент, тот получает те же возможности, что и `debug_agent`. Единственный способ получить корректное окружение — запустить компонент так, как он ожидает, и присоединить к нему отладчик.

После присоединения отладчика к процессу командами `zxdb` можно управлять процессом и исследовать его состояние. Вот короткий набор частых команд:

[Таблица команд `zxdb` — см. оригинал на fuchsia.dev.]

Упражнение: использование отладчика Fuchsia

В этом упражнении вы используете отладчик Fuchsia (`zxdb`), чтобы исследовать работающий экземпляр `echo-args` и понять причину падения.

Запуск эмулятора

Если у вас ещё нет запущенного экземпляра, запустите эмулятор:

- Запустите новый экземпляр эмулятора: `ffx emu start --headless`. По завершении запуска эмулятор печатает сообщение о готовности и возвращает управление.
- Запустите сервер пакетов, чтобы эмулятор мог загружать программные пакеты: `fx serve`

Запуск сессии отладки

После запуска эмулятора начните сессию отладки `zxdb` командой `ffx debug connect`:

```
ffx debug connect
```

```
Connecting (use "disconnect" to cancel)...  
Connected successfully.  
□ To get started, try "status" or "help".  
[zxdb]
```

После успешного подключения приглашение `zxdb` готово принимать команды.

Присоединение к компоненту

Перед запуском компонента настройте `zxdb` на присоединение к экземпляру `echo-args`. Это позволит отладчику присоединиться сразу после старта процесса:

```
[zxdb] attach echo-args
```

Установите точку останова на функции `greeting()`:

```
[zxdb] break greeting
```

С готовым отладчиком запустите новый экземпляр `echo-args`:

```
ffx component run /core/ffx-laboratory:echo-args  
fuchsia-pkg://fuchsia.com/echo-args#meta/echo-args.cm
```

Исследование сессии отладки

При достижении точки останова в `greeting()` выполнение останавливается, и отладчик ждёт команды. Командой `list` покажите, где сейчас приостановлено выполнение:

Команда `print` выведет состояние любых переменных в текущем кадре стека. Выведите текущее значение `names`:

Пройдите по функции `greeting()` несколько раз командой `next`:

```
[zxdb] next
```

Чтобы программа продолжила выполнение до конца, используйте команду `continue`:

```
[zxdb] continue
```

Выйдите из сессии отладки, чтобы вернуться в терминал:

```
[zxdb] exit
```

Внесение падающего кода

Далее вы добавите в `src/main.rs` код, вызывающий падение (`panic`) компонента. Сымитируйте это, добавив макрос `assert!(false)` сразу после сбора аргументов:

Выполните `fx build` снова, чтобы пересобрать компонент:

```
fx build
```

Запустите новую сессию отладки с `zxdb`:

```
ffx debug connect
```

Отладка падающего кадра стека

Настройте отладчик на присоединение к компоненту `echo-args`:

```
[zxdb] attach echo-args
```

Запустите новый экземпляр компонента:

```
ffx component run /core/ffx-laboratory:echo-args  
fuchsia-pkg://fuchsia.com/echo-args#meta/echo-args.cm
```

На этот раз отладчик обнаруживает, что было выброшено исключение. Командой `frame` исследуйте трассировку стека в точке падения:

Текущий кадр стека (`frame 0`) находится глубоко внутри системной библиотеки, но любой кадр можно исследовать, добавив перед командой номер кадра из трассировки.

Выведите значение аргументов в точке падения, передав номер кадра так:

Выйдите из сессии отладки, чтобы вернуться в терминал:

```
[zxdb] exit
```

Уничтожение экземпляра

Очистите экземпляр `echo-args` следующей командой:

```
ffx component destroy /core/ffx-laboratory:echo-args
```

Компоненты на уровне продукта

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/components/build-product>

Компоненты — основа ПО, работающего в пользовательском пространстве Fuchsia. Компоненты, которые вы рассматривали до сих пор, — часть базовой платформы, общей для всех продуктов на Fuchsia. Продукты Fuchsia состоят из компонентов платформы и специфичных для продукта элементов, отвечающих уникальным нуждам оборудования и пользовательского опыта. В этом разделе вы узнаете, как Fuchsia использует компоненты для настройки на уровне продукта.

Компоненты сессии (session components)

Сессии — это компоненты, инкапсулирующие пользовательский опыт продукта. Компонент сессии служит границей между платформой Fuchsia и взаимодействием с пользователем на уровне продукта. Каждый продукт Fuchsia определяет единственный экземпляр сессии как корень пользовательского опыта, который может управлять (а может и не управлять) дополнительными дочерними компонентами.

Компонент платформы `session_manager` запускает компонент сессии при загрузке и предлагает ему набор возможностей, необходимых для поддержки продукта, таких как управление окнами, обработка событий ввода или специальные возможности (accessibility).

Фреймворк драйверов (driver framework)

Подобно сессии, фреймворк драйверов Fuchsia позволяет разработчикам реализовывать специфичные для продукта драйверы устройств как компоненты. Одни компоненты-драйверы представляют контроллеры аппаратных интерфейсов, такие как PCI или USB, другие взаимодействуют с конечными устройствами, например контроллером Ethernet или клавиатурой.

По мере обнаружения или подключения устройств компонент платформы `driver_manager` запускает необходимые компоненты-драйверы, привязывает их к аппаратным интерфейсам и управляет их жизненным циклом.

Что дальше?

Поздравляем! Вы успешно собрали несколько базовых компонентов Fuchsia и интегрировали их в работающую систему.

В следующем модуле вы узнаете больше о том, как компоненты Fuchsia взаимодействуют друг с другом с помощью:

Интерфейсы Fuchsia

Часть IV. Интерфейсы FIDL

FIDL: обзор

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/fidl>

Во «Введении в Fuchsia» вы узнали, что Zircon предоставляет типы объектов ядра для поддержки межпроцессного взаимодействия (IPC) в Fuchsia. Эти типы определяют конкретные механизмы обмена данными между процессами. В этой схеме каналы Zircon обеспечивают асинхронный транспорт на основе сообщений, способный передавать как данные, так и набор дескрипторов для предоставления прав доступа.

Каналы Zircon — основа более высокоуровневых взаимодействий, описываемых языком определения интерфейсов Fuchsia (FIDL) — языком для описания IPC-протоколов программ Fuchsia. FIDL позволяет разнородным клиентам и серверам взаимодействовать, задавая набор семантических правил и форматов сериализации поверх канала.

Программы работают с FIDL-протоколами через языковые привязки (bindings) и библиотеки, генерируемые компилятором FIDL как слой абстракции над сложностью IPC Zircon. Это вводит привычные идиомы программирования, такие как структурированные типы и синхронное выполнение. Компилятор генерирует привязки для каждого поддерживаемого языка, поэтому поставщикам не нужно сопровождать клиентские библиотеки.

Предварительные требования

- Введение в Fuchsia
- Сборка Fuchsia
- Компоненты Fuchsia

Что вы узнаете

- Язык определения интерфейсов Fuchsia (FIDL)
- Генерацию FIDL-привязок
- Экспонирование и потребление FIDL в компонентах
- Тестирование и отладку FIDL-интерфейса

Что вам понадобится

- Рабочая машина под управлением Linux
- Эмулятор Fuchsia (FEMU)
- Успешно собранная Fuchsia с конфигурацией `fx set workstation_eng.x64`

Язык FIDL

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/fidl/fidl>

Язык определения интерфейсов Fuchsia (FIDL) — язык для описания протоколов межпроцессного взаимодействия (IPC) программ Fuchsia. FIDL предоставляет

упрощённый синтаксис объявления, позволяющий поставщикам определять интерфейсы как протокол (protocol). Поддерживаемые типы данных включают целые числа, числа с плавающей точкой, булевы значения, строки и дескрипторы. Их можно объединять в более сложные массивы, векторы, структуры (structs), таблицы (tables) и объединения (unions).

Рассмотрим следующий пример FIDL-протокола для интерфейса Echo:

```
library fuchsia.examples;

const MAX_STRING_LENGTH uint64 = 32;

@discoverable
closed protocol Echo {
    strict EchoString(struct {
        value string:MAX_STRING_LENGTH;
    }) -> (struct {
        response string:MAX_STRING_LENGTH;
    });
    strict SendString(struct {
        value string:MAX_STRING_LENGTH;
    });
    strict -> OnString(struct {
        response string:MAX_STRING_LENGTH;
    });
};
```

FIDL-протоколы описывают набор методов, вызываемых отправкой сообщений по каналу. Сообщения канала по своей природе асинхронны: отправитель и получатель работают независимо друг от друга. Методы FIDL вводят более высокоуровневую семантику, позволяющую писать более идиоматичный код на стороне клиента и сервера FIDL-транзакции.

FIDL поддерживает следующие типы методов:

- Двусторонние (two-way) методы: типичный вызов с необязательными параметрами и типом возврата, заданным после оператора `->`. Двусторонние методы блокируются до получения ответа. В примере Echo метод `EchoString()` — двусторонний.
- Односторонние (one-way) методы: асинхронные вызовы, возвращающиеся сразу, без ожидания ответа. Методы без объявленного типа возврата считаются односторонними со стороны клиента. В примере Echo метод `SendString()` — односторонний.
- События (events): при необходимости сервер может отправлять клиенту незапрошенные сообщения — события. События объявляют имя метода на стороне возврата оператора `->`. В примере Echo метод `OnString()` — событие.

Создание FIDL-библиотеки

FIDL-библиотеки группируют FIDL-файлы. Библиотека служит пространством имён для содержащихся в ней протоколов, и FIDL-файлы могут неявно обращаться ко всем другим объявлениям в той же библиотеке. Объявления из другой библиотеки FIDL-файл должен импортировать.

Система сборки Fuchsia предоставляет цель сборки `fidl()` для компиляции FIDL-файлов в библиотеку. Имя цели библиотеки должно совпадать с объявлениями `library` в каждом файле. См. пример `BUILD.gn` для библиотеки `fuchsia.examples`:

```
# Import the fidl GN template.
import("//build/fidl/fidl.gni")

# Define a FIDL library target.
fidl("fuchsia.examples") {
  # FIDL source files contained in library
  sources = [
    "echo.fidl",
  ]
}
```

На этапе сборки фронтенд-компилятор FIDL (`fidlc`) проверяет и компилирует файлы библиотеки в промежуточное представление JSON (JSON IR). Этот формат JSON IR — основа для FIDL-привязок.

Генерация FIDL-привязок

Компоненты потребляют FIDL-протоколы через генерируемый код — FIDL-привязки. Привязки кодируют и декодируют запросы и ответы как FIDL-сообщения и передают их по нижележащему IPC-каналу. Языковые библиотеки привязок предоставляют обёртки над этими структурами, чтобы согласовать взаимодействие с привычными идиомами программирования.

Клиентский интерфейс (иногда называемый прокси) выполняет преобразование между высокоуровневыми вызовами функций и FIDL-сообщениями. На стороне сервера привязки обрабатывают входящие сообщения-запросы и доставляют их через абстрактный интерфейс, который реализуют компоненты.

На этапе сборки бэкенд-инструменты `fidlgen` генерируют привязки для поддерживаемых языков из библиотеки JSON IR, созданной `fidlc`. Например, `fidlgen_rust` генерирует привязки на Rust из JSON IR.

Цель библиотеки `fidl()` создаёт отдельные цели привязок для каждого поддерживаемого языка. В силу природы GN эти привязки не генерируются при сборке, если не включены как зависимость.

См. пример фрагмента `BUILD.gn`, включающего цель сгенерированных привязок для библиотеки `fuchsia.examples`:

Упражнение: FIDL-библиотека Echo

В этом разделе вы определите новую FIDL-библиотеку с протоколом `Echo`, содержащим единственный метод, который возвращает строковые значения вызывающей стороне.

Начните с создания нового каталога для цели FIDL-библиотеки:

```
mkdir -p vendor/fuchsia-codelab/echo-fidl
```

Создайте следующую структуру файлов и каталогов в каталоге нового проекта:

```
//vendor/fuchsia-codelab/echo-fidl
| - BUILD.gn
| - echo.fidl
```

Добавьте новый файл FIDL-интерфейса `echo.fidl` со следующим содержимым:

```
library fidl.examples.routing.echo;

const MAX_STRING_LENGTH uint64 = 64;

@discoverable
closed protocol Echo {
  /// Returns the input.
  strict EchoString(struct {
    value string:<MAX_STRING_LENGTH, optional>;
  }) -> (struct {
    response string:<MAX_STRING_LENGTH, optional>;
  });
};
```

`EchoString` — двусторонний метод, который принимает необязательное (nullable) строковое значение и возвращает его же.

Добавьте файл `BUILD.gn` со следующим содержимым, чтобы объявить цель библиотеки:

```
import("//build/fidl/fidl.gni")
fidl("echo") {
  name = "fidl.examples.routing.echo"

  sources = [ "echo.fidl" ]

  enable_hlcpp = true
}
```

Добавьте цель библиотеки в конфигурацию сборки:

Изучение FIDL-привязок

GN-цель `fidl()` компилирует FIDL-интерфейс и генерирует дополнительные цели сборки для привязок на разных языках. Чтобы изучить привязки, нужно скомпилировать отдельные цели.

Скомпилируйте привязки `fidl.examples.routing.echo`:

С помощью GN найдите сгенерированные исходные файлы цели и откройте их в редакторе:

Изучите содержимое этих файлов. Ниже — краткое описание некоторых ключевых сгенерированных интерфейсов:

Инструментарий FIDL генерирует клиентские привязки, асинхронные по умолчанию. Это значит, что клиентские методы сразу возвращают управление вызывающей стороне, а ответы доставляются через колбэк или `future`. Синхронные клиенты предоставляют упрощённый API, гарантирующий доставку ответа до возврата из клиентского метода.

Синхронные клиенты доступны не во всех поддерживаемых языках. Подробнее см. спецификации для выбранного языка в «[Bindings Reference](#)».

Соединение компонентов

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/fidl/connecting-components>

Дескриптор протокола (protocol handle) — это общеизвестный объект, предоставляющий реализацию FIDL-протокола, который обнаруживается через пространства имён компонентов. Фреймворк компонентов обеспечивает обнаружение протоколов между компонентами через возможности. Маршрутизация возможностей описывает, какой компонент должен быть поставщиком для конкретного клиента. После определения нужных компонентов менеджер компонентов устанавливает соединения между ними, используя дескрипторы из пространства имён каждого компонента.

Рассмотрим следующий пример для протокола `fuchsia.example.Foo`:

Схема выделяет основные элементы, участвующие в установке соединения:

- Компонент-поставщик статически объявляет протокол в разделе `capabilities` манифеста. Это позволяет фреймворку выполнять маршрутизацию возможностей.
- Компонент-клиент статически запрашивает протокол в разделе `use` манифеста. При успешной маршрутизации это создаёт запись протокола `/svc/fuchsia.example.Foo` в пространстве имён клиента.
- Код поставщика публикует реализацию во время выполнения. Это создаёт запись протокола `/svc/fuchsia.example.Foo` в исходящем каталоге поставщика.
- Код клиента подключается к дескриптору протокола во время выполнения. Это открывает FIDL-соединение с реализацией, работающей в компоненте-поставщике.

Публикация реализации протокола

Компоненты, реализующие FIDL-протокол, объявляют и экспонируют его как возможность в своём манифесте. Это позволяет фреймворку выполнять маршрутизацию возможности от этого компонента к другим в топологии, которые её запрашивают.

```
{
  // ...
  capabilities: [
    { protocol: "fuchsia.example.Foo" },
  ],
  expose: [
    {
      protocol: "fuchsia.example.Foo",
      from: "self",
    },
  ],
}
```

Маршрутизация возможностей описывает права доступа к протоколу, но не устанавливает необходимые конечные точки соединения. Компоненты должны опубликовать реализацию как дескриптор `/svc/` в исходящем каталоге, используя протокол `fuchsia.io`. Сгенерированные FIDL-привязки оборачивают этот дескриптор и позволяют поставщику присоединить дескриптор запроса, чтобы начать получать

FIDL-сообщения.

Подключение к реализации протокола

Компоненты-клиенты объявляют протокол как требуемую возможность в манифесте. Это позволяет фреймворку определить, имеет ли компонент права на доступ к реализации протокола. Если допустимый маршрут существует, пространство имён компонента содержит соответствующий дескриптор `/svc/`.

```
{
  // ...
  use: [
    { protocol: "fuchsia.example.Foo" },
  ],
}
```

Компонент-клиент использует протокол `fuchsia.io`, чтобы установить соединение с реализацией протокола и открыть канал. Сгенерированные FIDL-привязки оборачивают этот канал и позволяют клиенту начать отправку сообщений поставщику.

Упражнение: сервер и клиент Echo

В этом разделе вы используете сгенерированные FIDL-привязки для `fidl.examples.routing.echo`, чтобы реализовать компоненты клиента и сервера на Rust.

Запуск эмулятора

Если у вас ещё нет запущенного экземпляра, запустите эмулятор:

- Запустите новый экземпляр эмулятора: `ffx emu start --headless`. По завершении запуска эмулятор печатает сообщение о готовности и возвращает управление.
- Запустите сервер пакетов, чтобы эмулятор мог загружать программные пакеты: `fx serve`

Создание компонента-сервера

Начните с создания нового проекта компонента для реализации сервера `echo`. Этот компонент будет обслуживать протокол `Echo` и обрабатывать входящие запросы.

Создайте каркас проекта для нового компонента `echo-server` в каталоге `//vendor/fuchsia-codelab`:

```
mkdir -p vendor/fuchsia-codelab/echo-server
```

Создайте следующую структуру файлов и каталогов в каталоге нового проекта:

Добавьте следующие правила сборки в файл `BUILD.gn`, чтобы собрать и упаковать компонент-сервер:

Объявите протокол `Echo` как возможность, предоставляемую сервером, и экспонируйте его для использования родительским `realm`:

Реализация сервера

Откройте главный исходный файл и замените операторы импорта следующим кодом:

Добавьте следующий код в `main()`, чтобы обслуживать протокол Echo:

Добавьте следующий код для реализации обработчика запросов протокола:

Эта реализация просто «возвращает эхом» то же строковое значение из запроса в полезной нагрузке ответа.

Создание компонента-клиента

Создайте ещё один новый проект компонента для реализации клиента echo. Этот компонент будет подключаться к реализации протокола и отправлять запросы.

Создайте каркас проекта для нового компонента echo-client в каталоге `//vendor/fuchsia-codelab/`:

```
mkdir -p vendor/fuchsia-codelab/echo-client
```

Создайте следующую структуру файлов и каталогов в каталоге нового проекта:

Добавьте следующие правила сборки в файл `BUILD.gn`, чтобы собрать и упаковать компонент-клиент:

Настройте манифест клиента, чтобы запросить возможность `fidl.examples.routing.echo.Echo`, экспонированную сервером:

Реализация клиента

Подобно `echo-args`, клиент передаёт аргументы программы как сообщение серверу. Добавьте следующие аргументы программы в `echo_client.cml`:

Откройте главный исходный файл и замените операторы импорта следующим кодом:

Добавьте следующий код в `main()`, чтобы подключиться к протоколу Echo и отправить запрос:

Интеграция компонентов

Возможности, предоставляемые сервером, должны быть маршрутизированы клиенту через фреймворк компонентов. Для этого вы реализуете компонент `realm`, который выступит родителем и будет управлять маршрутизацией возможностей.

Создайте новый каталог проекта для определения продукта `realm`:

```
mkdir -p vendor/fuchsia-codelab/echo-realm
```

Создайте следующую структуру файлов и каталогов в каталоге нового проекта:

```
//vendor/fuchsia-codelab/echo-realm
| - BUILD.gn
| - meta
|   | - echo_realm.cml
```

Создайте новый файл манифеста компонента `meta/echo_realm.cml` со следующим содержимым:

`echo-realm/meta/echo_realm.cml`:

```
{
  // Two children: a server and client.
  children: [
    {
      name: "echo_server",
      url: "#meta/echo_server.cm",
    },
    {
      name: "echo_client",
      url: "#meta/echo_client.cm",
    },
  ],
  offer: [
    // Route Echo protocol from server to client.
    {
      protocol: "fidl.examples.routing.echo.Echo",
      from: "#echo_server",
      to: "#echo_client",
    },

    // Route diagnostics protocols to both children.
    {
      dictionary: "diagnostics",
      from: "parent",
      to: [
        "#echo_client",
        "#echo_server",
      ],
    },
  ],
}
}
```

Так создаётся realm компонента с сервером и клиентом как дочерними компонентами, и возможность-протокол `fidl.examples.routing.echo.Echo` маршрутизируется клиенту.

Добавьте файл `BUILD.gn`, чтобы создать цель сборки для компонента `realm`: `echo-realm/BUILD.gn`:

```
import("//build/components.gni")

fuchsia_component("echo_realm") {
  manifest = "meta/echo_realm.cml"
}

fuchsia_package("echo-realm") {
  deps = [
    ":echo_realm",
    "//vendor/fuchsia-codelab/echo-server:component",
    "//vendor/fuchsia-codelab/echo-client:component",
  ]
}
```

Обновите конфигурацию сборки, чтобы включить новые компоненты:

```
fx set workstation_eng.x64 \
  --with //vendor/fuchsia-codelab/echo-server \
  --with //vendor/fuchsia-codelab/echo-client \
  --with //vendor/fuchsia-codelab/echo-realm
```

Выполните `fx build` снова, чтобы собрать компоненты:

```
fx build
```


Добавление компонентов в топологию

Вы добавите компонент в `ffx-laboratory` — ограниченную коллекцию для разработки внутри `realm core` продукта. Коллекции позволяют динамически создавать и уничтожать компоненты во время выполнения.

Создайте экземпляры компонентов, передав URL компонента `echo-realm` и подходящий `moniker` внутри `ffx-laboratory` команде `ffx component create`:

```
ffx component create /core/ffx-laboratory:echo-realm \
  fuchsia-pkg://fuchsia.com/echo-realm#meta/echo_realm.cm
```

Затем разрешите компонент `echo-realm` командой `ffx component resolve`:

```
ffx component resolve /core/ffx-laboratory:echo-realm
```

Проверьте, что экземпляры сервера и клиента также созданы как дочерние компоненты, командой `ffx component show`:

```
ffx component show echo
```

```
Moniker: /core/ffx-laboratory:echo-realm/echo_client
  URL: #meta/echo_client.cm
  Type: CML static component
Component State: Unresolved
Execution State: Stopped

Moniker: /core/ffx-laboratory:echo-realm/echo_server
  URL: #meta/echo_server.cm
  Type: CML static component
Component State: Unresolved
Execution State: Stopped

Moniker: /core/ffx-laboratory:echo-realm
  URL: fuchsia-pkg://fuchsia.com/echo-realm#meta/echo_realm.cm
  Type: CML dynamic component
Component State: Resolved
Execution State: Stopped
Merkle root: 666c40477785f89b0ace22b30d65f1338f1d308ecceacb0f65f5140baa889e1b
```

Проверка взаимодействия компонентов

Запустите существующий экземпляр компонента-клиента командой `ffx component start`:

```
ffx component start /core/ffx-laboratory:echo-realm/echo_client
```

Откройте другое окно терминала и проверьте вывод логов компонента-клиента:

```
ffx log --filter echo
```

В логах устройства вы должны увидеть следующий вывод:

```
[echo_client][I] Server response: Hello, Fuchsia!
```

Компонент-сервер запускается, как только клиент подключается к возможности `fidl.examples.routing.echo.Echo`, и продолжает работать, обслуживая дополнительные

FIDL-запросы.

Командой `ffx component show` посмотрите на работающий сервер `echo` в дереве экземпляров компонентов:

```
ffx component show echo_server
```

```
Moniker: /core/ffx-laboratory:echo-realm/echo_server
URL: #meta/echo_server.cm
Type: CML static component
Component State: Resolved
Incoming Capabilities: fuchsia.logger.LogSink
Exposed Capabilities: diagnostics
                    fidl.examples.routing.echo.Echo
Execution State: Running
Job ID: 474691
Process ID: 474712
Running for: 2026280474361 ticks
Merkle root: 666c40477785f89b0ace22b30d65f1338f1d308ecceacb0f65f5140baa889e1b
Outgoing Capabilities: diagnostics
                    fidl.examples.routing.echo.Echo
```

Уничтожение экземпляра

Очистите экземпляр `echo-realm` следующей командой:

```
ffx component destroy /core/ffx-laboratory:echo-realm
```

Интеграционное тестирование

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/fidl/integration-testing>

Интеграционное тестирование сосредоточено на проверке поведения вашего компонента при взаимодействии с другими компонентами системы. Поэтому интеграционные тесты обычно собираются отдельно от основного компонента и могут объявлять тестируемый компонент и другие зависимости как детей. В зависимости от характера теста компоненты-зависимости могут предоставляться как заглушки (mocks или stubs), чтобы тест-кейсы оставались герметичными.

Тестовые компоненты

Ниже — пример манифеста для простого интеграционного тестового компонента:

Это объявление тестового компонента содержит следующие ключевые элементы:

- Включение (include) нужного языкового шарда раннера тестов. Это позволяет менеджеру тестов корректно выполнить набор тестов.
- Перечисление тестируемого компонента и зависимых компонентов как детей.
- Маршрутизацию необходимых возможностей между компонентами в `realm` тестов.

Система сборки Fuchsia предоставляет GN-цель `fuchsia_test_package()` для отдельных тестовых компонентов, таких как интеграционные тесты. Это правило позволяет объявлять компоненты с тестами отдельно от тех, что нужны как зависимости, и описывает окружение целевого устройства, где должны выполняться тесты.

Вот пример того, как приведённый выше интеграционный тест мог бы быть включён в файл BUILD.gn:

```
import("//build/components.gni")
...

// Component under test
fuchsia_component("foo_client") {
  deps = [ ... ]
  manifest = "meta/foo_client.cml"
}

// Test dependencies
fuchsia_component("mock_service") {
  deps = [ ... ]
  manifest = "meta/mock_service.cml"
  testonly = true
}

// Component containing integration tests
fuchsia_component("integration_tests") {
  deps = [ ":bin_test" ]
  manifest = "meta/integration_tests.cml"
  testonly = true
}

fuchsia_test_package("hello-world-tests") {
  test_components = [ "integration_tests" ]
  deps = [
    ":foo_client",
    ":mock_service",
  ]
}
```

Упражнение: интеграционный тест сервера Echo

В этом упражнении вы добавите интеграционный тестовый компонент, чтобы проверить интерфейс FIDL-протокола компонента `echo_server` с помощью Test Runner Framework, и запустите эти тесты в окружении FEMU.

Добавление интеграционного тестового компонента

Для начала создайте каркас проекта для нового интеграционного тестового компонента `echo-integration` в каталоге `//vendor/fuchsia-codelab`:

```
mkdir -p vendor/fuchsia-codelab/echo-integration
```

Создайте следующую структуру файлов и каталогов в каталоге нового проекта:

Обновление манифеста тестового компонента

Манифест тестового компонента применяет базовые зависимости, такие как `test_runners`. Обновите файл `echo_integration.cml`, чтобы объявить компонент `echo-server` как ребёнка и маршрутизировать возможность-протокол `Echo` к тестовому компоненту.

Обратите внимание, что экземпляр `echo-server` берётся из того же пакета, что и интеграционный тест. Такая практика способствует герметичности тестовых пакетов, избегая зависимостей от компонентов из других пакетов.

Реализация интеграционного теста

Интеграционный тест подключается к протоколу Echo, экспонированному echo-server, так же, как компонент-клиент, отправляет строковый запрос и проверяет ожидаемый ответ.

Добавьте следующий код для реализации интеграционного теста:

Добавьте следующие правила сборки в файл BUILD.gn, чтобы собрать и упаковать интеграционный тестовый компонент вместе с зависимостями сервера echo:

Обновление конфигурации сборки

Добавьте пакет интеграционного теста в конфигурацию сборки:

```
fx set workstation_eng.x64 \
  --with //vendor/fuchsia-codelab/echo-server \
  --with //vendor/fuchsia-codelab/echo-client \
  --with //vendor/fuchsia-codelab/echo-realm \
  --with //vendor/fuchsia-codelab/echo-integration:tests
```

Выполните `fx build` и убедитесь, что сборка завершается успешно:

```
fx build
```

Запуск интеграционного теста

Правило `fuchsia_test_package()` генерирует пакет с тестовым компонентом и его зависимостями. Интеграционный тестовый компонент имеет следующий URL:

```
fuchsia-pkg://fuchsia.com/echo-integration-tests#meta/echo_integration.cm
```

Командой `ffx test` выполните интеграционные тесты. Убедитесь, что тесты проходят:

```
ffx test run \
  fuchsia-pkg://fuchsia.com/echo-integration-tests#meta/echo_integration.cm
```

Диагностика и мониторинг

Источник: <https://fuchsia.dev/fuchsia-src/get-started/learn/fidl/diagnostics-monitoring>

Подключение к FIDL-протоколам внутри компонентов — это сочетание маршрутизации возможностей и обслуживания каталогов. Поэтому диагностика проблем с подключением может охватывать несколько уровней:

- Клиент запрашивает возможность-протокол в своём манифесте.
- Поставщик экспонирует возможность-протокол в своём манифесте.
- Топология компонентов маршрутизирует возможность от поставщика к клиенту.
- Поставщик обслуживает протокол на правильном дескрипторе.
- Клиент пытается подключиться к правильному дескриптору протокола.

В этом разделе вы изучите API и инструменты, помогающие находить и устранять проблемы с соединениями компонентов и отслеживать долгосрочное здоровье ваших компонентов.

Мониторинг FIDL-соединений

Инструмент `fidlcat` позволяет отслеживать и отлаживать FIDL-соединения, трассируя отдельные FIDL-сообщения, отправляемые и получаемые вашим компонентом. Подобно отладчику Fuchsia (`zxdb`), `fidlcat` подключается к работающему компоненту `debug_agent` на целевом устройстве и отслеживает работающие процессы.

Настройка сессии мониторинга включает следующие высокоуровневые шаги:

- Запустить компонент `debug_agent` на целевом устройстве.
- Запустить клиент `fidlcat` и подключиться к целевому устройству.

Простейший способ начать сессию отладки FIDL — команда `ffx debug fidl`, которая делает всё это в контексте вашей локальной сборки Fuchsia. Однако эти шаги можно выполнить и вручную.

Ниже — пример сообщения `fidlcat` для запроса FIDL-протокола. Вывод трассировки содержит полезную информацию о каждом преобразовании, включая:

- Имя компонента или процесса
- Вызванный системный вызов
- Имя FIDL-библиотеки, протокола и метода
- Полезную нагрузку сообщения с параметрами или возвращаемыми значениями

```
echo-client.cm 256109:256122 zx_channel_read(handle:handle: e4c7c57f, options:uint32: 0,
  num_bytes:uint32: 48, num_handles:uint32: 0)
-> ZX_OK
  received response fidl.examples.echo/Echo.EchoString = {
    response: string = "hello world!"
  }
```

Использование Inspect

Инспекция компонентов (Component Inspection) позволяет компонентам Fuchsia экспонировать структурированную диагностическую информацию о себе через API `Inspect`. Fuchsia предоставляет эту информацию через инструменты разработчика и отчёты об ошибках, помогая диагностировать проблемы или отслеживать производительность.

Компоненты экспонируют метрики инспекции как дерево именованных узлов (Nodes), каждый из которых содержит набор свойств (Properties) в виде пар ключ/значение. Свойства поддерживают разные числовые, строковые и массивные типы данных. Библиотеки `inspector` предоставляют интерфейс к корневому узлу компонента, к которому можно прикреплять дополнительные интересные свойства вашего приложения.

Текущий набор метрик, опубликованных в `Inspect`, можно получить с помощью инструментов разработчика:

- `ffx inspect`: позволяет интерактивно запрашивать состояние `Inspect` с помощью селекторов компонентов. Удобно для отладки компонентов в процессе разработки.

- `ffx target snapshot`: захватывает отладочный снимок (snapshot) всей системы, содержащий данные Inspect в формате JSON.

```
ffx inspect show core/foo-example
```

```
core/foo-example:
  metadata:
    filename = fuchsia.inspect.Tree
    component_url = fuchsia-pkg://fuchsia.com/foo-example#meta/foo-example.cm
    timestamp = 55457379176
  payload:
    root:
      version = 1.0
      request_metrics:
        request_count = 3
        error = timeout
```

Упражнение: мониторинг компонентов-поставщиков

В этом разделе вы используете инструменты диагностики для мониторинга здоровья и поведения компонента-сервера `echo`.

Запуск эмулятора

Если у вас ещё нет запущенного экземпляра, запустите эмулятор:

- Запустите новый экземпляр эмулятора: `ffx emu start --headless`. По завершении запуска эмулятор печатает сообщение о готовности и возвращает управление.
- Запустите сервер пакетов, чтобы эмулятор мог загружать программные пакеты: `fx serve`

Мониторинг FIDL-трафика

С помощью `fidlcat` можно отслеживать и отлаживать FIDL-соединения ваших компонентов. Запустите `ffx debug fidl` и настройте его на мониторинг компонента-сервера `echo`:

```
ffx debug fidl --remote-name echo_server.cm
```

```
Checking for debug agent on [fe80::d6c5:4526:c282:fb6%qemu]:2345.
Debug agent not found. Starting one.
INFO: [main.cc(238)] Connected to symbol server gs://fuchsia-artifacts-release/debug
INFO: [main.cc(122)] Connecting to port 2345 on fe80::d6c5:4526:c282:fb6%qemu...
INFO: [main.cc(92)] Connected!
```

Иницилируйте FIDL-соединение с сервером, запустив экземпляр клиента `echo`:

```
ffx component start /core/ffx-laboratory:echo-realm/echo_client
```

Клиент привязывается к компоненту-серверу и общается по FIDL-протоколу `Echo`. Просмотрите вывод `ffx debug fidl`, чтобы увидеть список FIDL-транзакций, обработанных сервером `echo`:

Monitoring echo_server.cm

```
echo_server.cm 58694:58696 zx_channel_read_etc(handle: handle = fb9b5273, options: uint32 = 0,
  num_bytes: uint32 = 512, num_handles: uint32 = 4)
-> ZX_OK
  received request fuchsia.io/Directory.Open = { flags: uint32 = 3, mode: uint32 = 493, path:
  string = "svc/fidl.examples.routing.echo.Echo", object: handle =
  Channel:f93b597b(ZX_RIGHT_TRANSFER | ZX_RIGHT_READ | ZX_RIGHT_WRITE | ZX_RIGHT_SIGNAL |
  ZX_RIGHT_SIGNAL_PEER | ZX_RIGHT_WAIT |
  ZX_RIGHT_INSPECT)(channel:0:svc/fidl.examples.routing.echo.Echo) }

echo_server.cm 58694:58696 zx_channel_read_etc(handle: handle =
  Channel:f93b597b(channel:0:svc/fidl.examples.routing.echo.Echo), options: uint32 = 0,
  num_bytes: uint32 = 512, num_handles: uint32 = 4)
-> ZX_OK
  received request fidl.examples.routing.echo/Echo.EchoString = { value: string = "Hello,
  Fuchsia" }

echo_server.cm 58694:58696 zx_channel_write_etc(handle: handle =
  Channel:f93b597b(channel:0:svc/fidl.examples.routing.echo.Echo), options: uint32 = 0)
  sent response fidl.examples.routing.echo/Echo.EchoString = { response: string = "Hello,
  Fuchsia" }
-> ZX_OK

echo_server.cm 58694:58696 zx_channel_read_etc(handle: handle =
  Channel:f93b597b(channel:0:svc/fidl.examples.routing.echo.Echo), options: uint32 = 0,
  num_bytes: uint32 = 512, num_handles: uint32 = 4)
-> ZX_ERR_PEER_CLOSED

echo_server.cm 58694:58696 zx_handle_close(handle: handle =
  Channel:f93b597b(channel:0:svc/fidl.examples.routing.echo.Echo))
-> ZX_OK
```

Обратите внимание на последовательность событий:

- Канал к реализации протокола открывается на `svc/fidl.examples.routing.echo.Echo`.
- Сервер получает запрос `Echo.EchoString` по открытому каналу, содержащий строковую полезную нагрузку, отправленную клиентом.
- Сервер отправляет соответствующий ответ с той же строковой полезной нагрузкой.
- Канал закрывается.

Трассируя FIDL-соединения между компонентами, `fidlcat` позволяет находить и диагностировать потенциальные проблемы, такие как неудачные соединения или некорректные полезные нагрузки данных.

Понаблюдайте за FIDL-соединениями в `echo-client.cm` с помощью `fidlcat`. Видите ли вы что-то отличное от сообщений, записанных для сервера `echo`?

Добавление учёта запросов

Инспекция компонентов позволяет публиковать диагностическую информацию из ваших компонентов для упрощения отладки. С помощью API `Inspect` вы будете отслеживать некоторую статистику использования компонента-сервера `echo`.

Обновите обработчик запросов `echo_server`, чтобы он принимал новую структуру с числовыми `Inspect`-свойствами для счётчика запросов и обработанных байтов.

Обработчик увеличивает эти свойства при каждом входящем запросе:

Добавьте следующий код в `main()`, чтобы инициализировать `Inspect`-свойства и передать их обновлённому обработчику:

Шаблон компонента включает код, задающий статус здоровья компонента. Это стандартизированная метрика инспекции здоровья, удобный способ сообщить, работает ли ваш компонент хорошо или испытывает проблему. Этот статус публикуется под `fuchsia.inspect.Health` в дереве `Inspect`.

Подробнее об этой метрике см. «Health check».

Наконец, обновите импорты, добавив новые библиотеки `Inspect`:

Выполните `fx build` снова, чтобы пересобрать компонент:

```
fx build
```

Проверка данных `Inspect`

Остановите текущий экземпляр компонента `echo-server`. Это позволит компоненту при следующем запуске разрешить последнюю версию с сервера пакетов.

```
ffx component stop /core/ffx-laboratory:echo-realm/echo_server
```

Запустите компонент-клиент `echo` несколько раз. Это увеличивает счётчик запросов в `echo-server` при каждом соединении:

```
ffx component start /core/ffx-laboratory:echo-realm/echo_client
ffx component start /core/ffx-laboratory:echo-realm/echo_client
ffx component start /core/ffx-laboratory:echo-realm/echo_client
```

Просмотрите доступные данные `Inspect` для компонента-сервера `echo` командой `ffx inspect`. Вы увидите значения счётчика запросов и обработанных байтов в дереве под корневым узлом рядом со статусом здоровья компонента:

```
ffx inspect show 'core/ffx-laboratory\:echo-realm/echo_server'
```

```
core/ffx-laboratory\:echo-realm/echo_server:
  metadata:
    filename = fuchsia.inspect.Tree
    component_url = #meta/echo_server.cm
    timestamp = 1476246046122
  payload:
    root:
      bytes_processed = 42
      total_requests = 3
      fuchsia.inspect.Health:
        start_timestamp_nanos = 1467828507317
        status = OK
```

Публикация информации о здоровье и поведении через `Inspect` позволяет наблюдать текущее состояние компонентов и диагностировать проблемы на устройствах в продакшене.

Командой `ffx target snapshot` захватите отладочный снимок с целевого устройства. Этот архив также содержит данные `Inspect` в формате `JSON`. Сможете ли вы найти те же данные сервера `echo`, что вернула `ffx inspect`?

Уничтожение экземпляра

Очистите экземпляр echo-realm следующей командой:

```
ffx component destroy /core/ffx-laboratory:echo-realm
```

Что дальше?

Поздравляем! Вы успешно построили IPC-интерфейс Fuchsia с помощью FIDL и соединили два компонента через этот интерфейс.

Вы прошли все модули этого курса! Поднимите новое понимание на следующий уровень и погрузитесь глубже в:

Концепции Fuchsia